TWIN-64
Architecture Document

Helmut Fieres
November 17, 2025

Contents

List of Figures

List of Tables

Part I

Introduction

Introduction

Designers of CPUs in the seventies and early eighties almost all used a microcoded approach with hundreds of instructions. RISC was just beginning to emerge. Registers were typically not fully general-purpose but often had special functions or meanings, and many instructions were rather complicated, though designers believed them useful. Compiler writers, however, often used only a small subset, ignoring these complex instructions because they were too specialized. In the late eighties and nineties the shift moved to RISC-based designs, with large sets of registers, fixed instruction word lengths, large virtual memory address ranges, and instructions that were in general much more pipeline-friendly. Today, processors are highly complex, with superscalar deep pipelines and multilevel cache hierarchiesjust a few of the hallmarks of modern CPU design. CPU designs have become so large and complex that no single person can completely understand them. But what if there were a simple design that one person could fully understand?

Twin-64

Welcome to Twin-64. Twin-64 is a simple 64-bit CPU with a registermemory model and segmented virtual memory. The design is heavily influenced by Hewlett-Packards PA_RISC architecture, which was initially a 32-bit RISC-style load/store machine. Almost all of the key architectural features come from there. However, other processorssuch as the Data General MV8000, the DEC Alpha, and the IBM PowerPCwere also influential or at least investigated for their design choices. In addition, the Blitz-64 architecture was studied. Blitz-64, a design developed at Portland State University, offered an interesting approach in that it only supported 64-bit signed arithmetic, avoiding common errors from mixing signed and unsigned arithmetic.

Where does the name Twin-64 come from? Obviously, the 64 indicates that the CPU is a 64-bit CPU. The Twin part will become clearer when implementations of the CPU are described in later chapters. One implementation represents a dual-issue instruction design with two ALUs for computation. Nevertheless, a very basic implementation could just offer single-issue, single-ALU execution as well.

Design Goals

While it is not a design goal to build an industry-strength, competitive CPU, the CPU should nevertheless be useful and largely follow established design practices. For example, instructions should not be invented just because they seem useful, but rather designed with the assembler and compilers in mind. Instruction set design is CPU design. Registermemory architectures were typically microcoded, complex-instruction machines. In contrast, Twin-64 instructions will be hardwired and are in general pipeline-friendly, avoiding data and control hazards and stalls where possible.

This book

This document serves as a comprehensive guide to the Twin-64 processor, its instruction set, and runtime environment. It is organized into several parts, each focusing on a key aspect of the system.

Part Two introduces the Twin-64 architecture, providing an overview of the overall system and processors structure, core components, and design philosophy. This sets the stage for understanding how instructions are executed and how data moves through the system.

Part Three presents the instruction set in detail. It covers computational, immediate, memory reference, branch, and system control instructions. These chapters serve as the definitive reference for programmers and designers alike, offering precise information on encoding, operation, and usage.

Part Four explores the I/O subsystem, detailing the mechanisms by which the processor communicates with external devices and handles input/output operations efficiently.

Part Five discusses the runtime environment and software conventions. It explains program interaction with Twin-64, including calling conventions, exception handling, and runtime support for higher-level languages.

Part Six examines processor design. While multiple design approaches are possible, this section provides a reference implementation that highlights key architectural concepts, practical trade-offs, and performance considerations.

Part Seven presents a reference implementation of the architecture and instruction set processor in a simulator environment.

Finally, the **Appendix** provides supplementary material, including additional design notes, tables, and clarifications to support the main text and offer deeper insight into the Twin-64 system.

Part II

Architecture

System Organization

This chapter introduces Twin-64 by highlighting the major components and design concepts.

A RegisterMemory Architecture

Modern RISC CPUs are typically load / store designs and feature a large number of general-purpose registers. Operations take place between registers. Twin-64 follows a slightly different route. It has fewer general registers, and in addition to registerregister computation, a register-memory model is also supported. However, there are no instructions that read from and write to memory in a single instruction cycle, since this would complicate the design considerably.

Twin-64 implements a **register-memory architecture** offering only a few addressing modes for the operand, combined with a fixed instruction word length. For most instructions one operand is a register, while the other is a flexible operand that can be an immediate, another register, or a memory address from which data is obtained or stored. The result is placed in the register, which is also the first operand. These types of machines are also called two-address machines.

In contrast to similar historical register-memory designs, Twin-64 has no operations that both read from and write to memory in the same instruction. For example, there is no increment memory instruction, since this would require two memory operations and is generally not pipeline-friendly. Memory data access is performed at the word, half-word, or byte level. Besides the implicit operand fetch in a computational instruction, there are dedicated load/store instructions. In addition to the registerimmediate and registermemory operand modes, one operand mode supports the three-operand model ($R_s = R_a \,\mathrm{OP}\, R_b$), specifying two source registers and a result register. This allows Twin-64 to support both registermemory and load/store operation styles.

System

A Twin-64 system is organized around a central system bus that interconnects the key components of the architecture. Processors, main memory modules, and I/O modules are all attached as peers to this bus, enabling uniform communication and data exchange. Larger configurations may also include I/O adapters, which bridge slower peripheral buses to the central system bus while preserving a consistent programming model.

Processors initiate memory and I/O transactions via the bus and can also receive external events such as interrupts or system resets through the same interface. This modular structure provides flexibility in system design, allowing configurations to scale from simple processormemory systems to complex multiprocessor environments with diverse I/O subsystems.

Processor

The Twin-64 processor implements the instruction set architecture and provides the computational resources of the system. It contains a general register set, control and status registers, and the program state word. To support virtual memory, each processor includes a Translation Lookaside Buffer (TLB) that translates virtual addresses into physical addresses. Instruction and data caches, while optional in principle, are essential for performance and tightly integrated with the memory interface. Regardless of configuration, all processor-generated addresses are virtual and must be resolved by the TLB before memory access.

Virtual Memory

Twin-64 offers a large global address range, organized in segments. A segment number and offset together form the global virtual address. Segments are also associated with access rights and protection identifiers. The CPU itself can run in user or privileged mode. Twin-64 always generates virtual addresses at the CPU level. The global virtual memory range is a 52-bit space organized into a 20-bit segment ID and a 32-bit byte offset. There are 2^{20} segments with 2^{32} bytes each.

Virtual addresses are translated to physical addresses by the TLB when memory is referenced. For memory management purposes, the virtual address range can also be viewed as a set of pages. A segment therefore contains 2^{18} pages of 16 KB each. A virtual page number, combining the segment and the page number within that segment, is a 2^{38} value. Address translations are made at the page level.

The first segments in the virtual address space are special in that they also represent the physical address range. A virtual address in that range bypasses the TLB and accesses physical memory directly.

A TLB can optionally support different page sizes in addition to the architected 16 KB page size. Supported sizes are 16 bytes, 256 KB, 4 MB, and 64 MB. It is very common for the operating system and data to be mapped in consecutive physical pages, which can then be covered by a larger page size.

Security and Protection Support

Twin-64 implements a protection model along two dimensions. The first dimension is **access type control** and privilege-level checking. Each page is associated with a page type. Page types include read-only, readwrite, execute, and gateway. There are two privilege levels: user and supervisor mode. Access type and privilege level together form the access control information, which is checked for each instruction and data memory access.

The second dimension of protection is the **protection ID**, which is the segment ID itself. Twin-64 allows a set of segment IDs to be recorded in processor control registers. For each access to a segment that has the protection check bit set, one of the protection ID control registers must match the segment ID. If not, a protection violation trap is raised.

Privilege Changes

Instructions can only access data or execute instructions when the privilege level matches the required privilege level. Two or four levels are the most common implementations. Changing between levels is often done with a kind of trap operation to higher-privilege software, for example the operating system kernel. However, traps are expensive, as they cause the CPU pipeline to be flushed when entering a trap handler.

Twin-64 implements gateways for privilege promotion. A special option on the unconditional branch instruction raises the privilege level when the instruction is executed on a gateway page, setting the privilege level to that of the gateway page. Returning from a higher privilege level to a code page with lower privilege level automatically resets the lower privilege level. A branch to a higher-privilege page that is not a gateway page results in a privilege violation trap.

Debugging Support

A fundamental requirement is the ability to set instruction and data breakpoints. An instruction breakpoint is triggered by encountering the `TRAP` instruction. The break instruction causes an instruction break trap and passes control to the trap handler. Since break instructions are normally not present in the instruction stream, they must be inserted dynamically in place of the instruction normally found at that address. Upon continuation, if the breakpoint is still valid, it must be reinserted after the original instruction is executed. To facilitate this mechanism, a bit in the status word provides an easy way to trap again after the instruction has been executed.

Data breakpoints are traps taken when a data reference to a page occurs. They do not modify the instruction stream. Depending on the data access, the trap may occur before or after the instruction that accesses the data. A write operation is trapped before the data is written, while a read instruction is trapped after the original instruction has executed.

Input/Output

Twin-64 implements a memory-mapped I/O architecture. A portion of the physical address space is reserved for I/O modules, which respond to memory load and store instructions directed to their assigned addresses. Standard page-level protection applies during address translation, ensuring that only privileged software (typically device drivers) can safely access I/O regions. User programs may be granted direct access to I/O by mapping a user-level I/O page into virtual memory, without compromising overall system integrity.

Direct I/O is performed using the standard load and store instructions to access I/O modules. These operations are entirely controlled by software and may be executed in user mode if the virtual mapping allows it.

Direct Memory Access (DMA) modules can transfer data between memory and I/O devices independently of the processor. DMA transfers operate on physical memory, requiring that the relevant pages be locked and protected from conflicting access by the operating system. DMA

transactions are initiated by issuing DMA commands, and command chaining is supported to enable complex or continuous data transfers without processor intervention.

Interrupt System

Interrupts and exceptions are events that occur asynchronously to instruction execution. Depending on the type, they occur either during the execution of an instruction or between instructions. An example of the former is an arithmetic overflow trap; an example of the latter is an external interrupt. Depending on the interrupt or exception type, the instruction is either restarted or execution continues with the next instruction.

Twin-64 implements a simple interrupt system. Each processor features an interrupt input register and an interrupt mask register. When an I/O module receives a command, it also receives information about which interrupt bit to set and which target module ID to send the interrupt to upon completion of the request. The interrupt data is compared to the interrupt mask register. If the particular bit is enabled and interrupts are globally enabled, the target processor enters the interrupt handler.

Since raising an interrupt is simply a matter of storing the interrupt request into the processors interrupt register, I/O modules as well as processors themselves can raise interrupts on a target processor. Processors can also send interrupts to their peers, enabling low-level processor-to-processor communication.

Processing Resources

This chapter describes the processing resources and data types in a Twin-64 system. Twin-64 provides a set of general-purpose registers, a set of control registers, and the Program State Word (PSW), which contains the current instruction address and processor status. The following figure presents a high-level overview of these register sets.

Figure 1: Software Accessible Registers

General registers are used for computation and address storage, control registers manage processor state, and the PSW holds the current instruction address along with the processor status.

General Registers

Twin-64 features a set of 16 general purposes registers. They are used for all computations including address arithmetic.

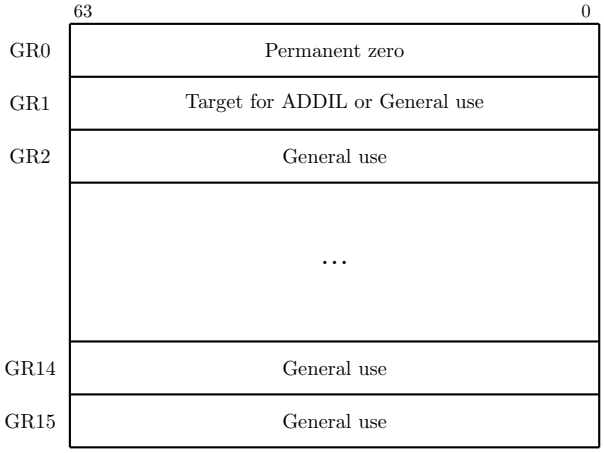


Figure 2: General registers

Some general registers have dedicated purposes. Register zero always returns a zero value when read, and writing to it has no effect. Although the CPU has only 16 general registers, implementing such a register greatly simplifies instruction design, for example by providing a

target when the result is not needed. General register one is a scratch register and also serves as an implicit target for certain instructions. Other general registers may have dedicated roles as defined by the runtime architecture conventions.

Control Registers

Twin-64 has 16 defined control registers, numbered `CR0` to `CR15`. Two instructions allow moving data between a control register and a general register in either direction. A move to a control register always transfers the full 64-bit word. A move from a control register to a general register transfers the value right-aligned to the size of the control register. The following figure depicts the control registers defined for Twin-64.

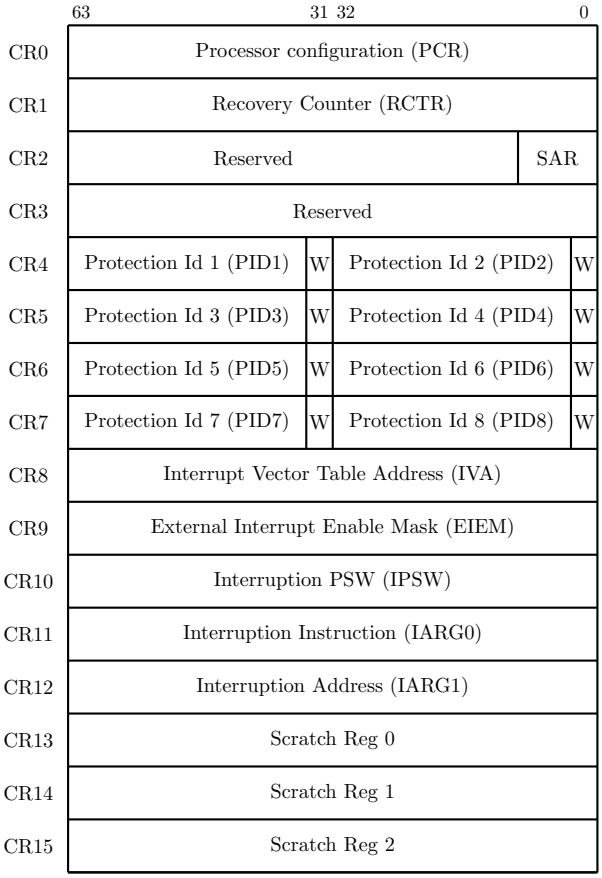


Figure 3: Control registers

Processor Configuration Register

The processor configuration register contains information about the current system and processor hardware setup. Some of its fields are read-only and set directly by the hardware, while others can be modified by processor-dependent code routines.

Table 1: Processor Configuration Register Fields

Field	Purpose
Memory Size (MSize)	The available physical memory in 4 Gbyte increments. The minimum size is 4 Gbyte, encoded as a zero. The maximum size is 64 Gbyte encoded as a 15.
Architecture Id (AId)	...
Processor Id Id (PId)	...
Processor Core Num	in a multiprocessor configuration, the unique processor number is assigned during system startup.
Flags (tbd)	e.g default endian, etc.
Cycles per millisecond	the cycle per millisecond specifies the number of processor cycles for a millisecond.

Recovery Counter

When enabled, the recovery counter decrements on each instruction cycle. When the leftmost bit becomes a one, a recovery trap is scheduled. The R bit in the processor status word is set, the recovery trap is enabled.

Shift Amount Register

The shift amount register (`SAR`) is used by the extract, deposit and double shift instruction when a dynamic position is specified in the instruction. The valid number range is zero to 63.

Protection Id Registers

Twin-64 allows to record a set of up to 8 protection Id values in the processor control registers `CR4` to `CR7` which are accessible to the currently executing process. The protection Id is identical to a segment Id. For each access to a segment that has the protection check bit set, one of the protection Id control registers must match the segment Id. If not, a protection violation trap is raised. The rightmost bit of each of the eight PIDs is the write disable (W) bit. When the bit is set, that PID cannot be used to grant write access. See also the chapter on addressing and access control.

Interrupt Vector Table Address

The Interruption Vector Table Address (IVA) control register, `CR8`, holds the base address of an array of service routines assigned to the different interruption classes. The lower 14 bits of the IVA are reserved and must be set to zero, meaning any address written to it must be aligned

to a page boundary. The IVA is always located in the first segment, segment 0. The array of interruption service routines is indexed by interruption numbers, as described in the chapter on interrupts.

External Interrupt Enable Mask

The External Interrupt Enable Mask (EIEM) is stored in `CR15)`. It is a 64-bit register, with each bit corresponding to one of the 64 external interrupts. A cleared bit in the EIEM masks the external interrupt associated with that position.

Interruption PSW

Upon an interrupt, the address of the interrupted instruction and the processor status bits are saved to this control register.

Interruption Argument Registers

The Interruption Argument Registers (`CR11` and `CR12`) are used to pass an instruction and a virtual address to an interruption handler. The values of these registers for each interruption class are described in the chapter on interrupts.

Scratch Registers

Control registers `CR13`, `CR14`, and C`CR15` are scratch registers used by interrupt handlers or similar routines. `CR13` and `CR14` are privileged and can only be accessed from code running at a privileged level, whereas `CR15` is accessible from any privilege level.

Program State Register

The processor state register contains the virtual address of the current instruction along with the processor status flags.

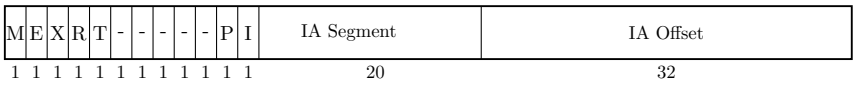


Figure 4: Program State Register

Instruction Address

The instruction address portion of the PSW registers contains the segment id and the byte offset into the segment for the currently executing instruction.

Processor Status

??? finalize the bits and position ....

Table 2: Status Field Bits

Bit	Purpose
M	High priority machine check. When set, high-priority machine checks are masked. This bit is set after an HPMC and cleared after all other interruptions.
E	Little endian access enable. When set, memory access is in little endian format, else big endian format. Endian support is an optional feature. If not implemented, all access is in big endian format.
R	Recovery counter enable. When set...
X	When set, the processor runs in privileged mode.
T	When set, taken branch traps are enabled. Any branch taken will result in a trap.
I	When set, external interrupts are enabled.
V	Reserved.
P	Protection identifier validation enable. When set instruction and data references are checked for valid protection identifiers (PIDs). The `PRB` instruction checks for valid PIDs without causing a trap.
D	Data memory break disable. The D-bit is cleared after the execution of each instruction, except for the `RFI` instruction which may set it. When set, data memory break traps are disabled. This bit allows a simple mechanism to trap on a data store and then proceed past the trapping instruction.

Data Types

The fundamental data type is a 64-bit signed integer, with the most significant bit on the left and the least significant bit on the right. All computations are performed on 64-bit signed integers. Memory accesses can be performed at the granularity of a double-word, word, halfword, or byte. Data loaded as a word, halfword, or byte is sign-extended to 64 bits. Store operations write the corresponding value to memory without modification.

All memory addresses are expressed as bytes addresses. Address computation uses a 32-bit unsigned math, i.e. numbers wrap around in a 32-bit space and never cross segment boundaries.

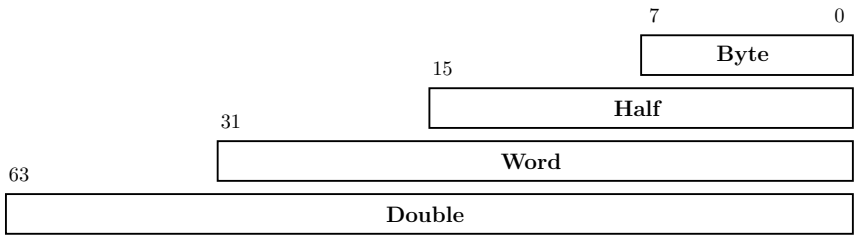


Figure 5: Data types

Byte Ordering and Alignment

Twin-64 is a big-endian machine. The following figure shows the memory access for load operations.

Figure 6: Big endian mode

The processor status register reserves a bit position, "E", for future support of mixed big and little endian memory access. Currently this bit is always set to zero.

Twin-64 enforces natural alignment for memory accesses. Byte, half-word, word, and double-word data types must be aligned to 1, 2, 4, and 8-byte boundaries respectively. If an access does not meet the required alignment, a data-alignment trap is raised.

Translation Lookaside Buffer

Virtual address translations are stored in a translation lookaside buffer (TLB). The TLB is essentially a cache of page table entries representing the virtual pages currently mapped to physical addresses. Depending on the hardware implementation, there may be a single combined TLB or separate instruction and data TLBs. Each TLB entry stores the virtual page number (VPN) along with its attributes and the corresponding physical page number (PPN). The current architecture supports a 36-bit physical address space, allowing a maximum of 64GB of memory.

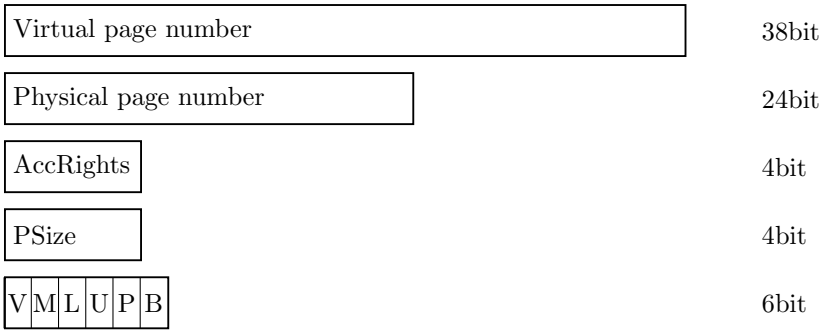


Figure 7: Tlb info

To reduce TLB processing overhead, the TLB is capable of storing virtual address ranges. There is support for a larger page sizes. The supported sizes are 4 Kbyte, 64 Kbyte, 1 MByte and 16Mbyte.

Table 3: TLB Fields

Field	**Purpose**
VPN	The virtual page number.
PPN	The physical page number.
Access Rights	Encodes the access type (read, write, and execute) and the required privilege level for the operation.
Page Size	4-bit field encoding the supported page size: `( 1U << ( pSize * 4 )) * T64_PAGE_SIZE` . The pSize number 4 to 15 are reserved and result in an undefined operation. A page size is not allowed to cross region boundaries. The virtual address must be aligned with selected the page size.
V	Indicates if the entry is valid.
M	When set, the page has been modified. This only applies to data TLBs or unified TLBs.
L	When set, the TLB entry is locked and will not be selected during replacement.
U	When set, the page represented by the TLB entry is not cached.
P	When set, the page has protection checking enabled.
B	When set, a branch instruction executed from this page will cause a branch-taken trap. This inly applies for instruction TLBs or unified TLBs.

The first segments, segment 0 to 15, are special in that they bypass the TLB entirely. A virtual address in the range 0x00_0000_0000 to 0x03_FFFF_FFFF directly maps to the physical address

of the same range. The address range represents physical memory and can only be accessed in privileged mode.

Caches

Twin-64 is a cache-visible architecture that supports both unified and separate instruction and data caches. While the hardware directly manages cache line insertion and replacement, software has explicit control over flushing and purging cache line entries. Twin-64 caches are virtually indexed and physically tagged. To accelerate memory access, the cache uses the virtual address to index into the arrays in parallel with the TLB lookup. A cache hit occurs when the physical address tag in the cache matches the physical page address from the TLB.

The architecture does not mandate specific cache line sizes, cache organization, or hierarchy models. Instead, it defines instructions for cache operations, such as deletion or flushing of cache lines.

Each TLB entry indicates whether the corresponding data should be cached. For segments 0 to 15, where the TLB is bypassed, the hardware determines the caching behavior. Memory address ranges are always cached, whereas the I/O address range is not cached.

Instruction Set

Twin-64 features a simple and efficient instruction set. All instructions are of fixed length, specifically a 32-bit word. The total number of instructions is relatively small, but many include execution options that make them highly versatile. Great care has been taken to ensure these options do not significantly complicate the pipeline or lengthen the data path.

Unlike CISC-style processors that support a wide variety of addressing modes, Twin-64 provides just two virtual memory addressing modes, both based on a simple baseoffset calculation. No indirection or mode requiring a memory operand to compute an address is part of the architecture.

The instruction set is organized into five groups:

- **Computational instructions:** Arithmetic, Boolean, and bit-manipulation operations such as extract and deposit. Twin-64 supports only 64-bit signed arithmetic. Any numeric overflow results in a trap.

- **Immediate instructions:** Instructions for constructing immediate values up to 64 bits. Several instructions embed immediate fields within the instruction word, but a full 64-bit constant requires a sequence of two or three instructions. Immediate instructions also support 32-bit address arithmetic, where calculations wrap around within the 32-bit offset field and never overflow into the segment identifier portion.

- **Memory reference instructions:** Load and store operations for both virtual and physical memory. These transfer data between memory and general registers in sizes of double–word, word, halfword, or byte. The design resembles a load/store architecture, but unlike

strict load/store machines, many Twin-64 instructions allow memory operands directly. No instruction, however, both reads and writes data memory in the same operation.

- **Control flow instructions:** Both unconditional and conditional branches are supported. Unconditional branches may optionally save the return address in a general-purpose register. Conditional branches compare two operands and transfer control if the condition is satisfied, eliminating the need for a separate compare instruction.

- **System control instructions:** Instructions for managing processor resources such as the TLB, cache, and control registers. Control registers can be accessed directly; the TLB can be updated with insert and remove operations; and the cache can be flushed or purged. Additional instructions generate traps or provide diagnostic functions for examining processor state. Most system instructions require execution in privileged mode.

The book section on the instruction set will describe each instruction group in detail, with formats, descriptions, and examples.

System Memory

Virtual Address Space

Twin-64 features a 52-bit virtual address range, organized into a 20-bit segment identifier and a 32-bit offset within that segment. The following figure illustrates the structure of a virtual address and how the segment is selected.

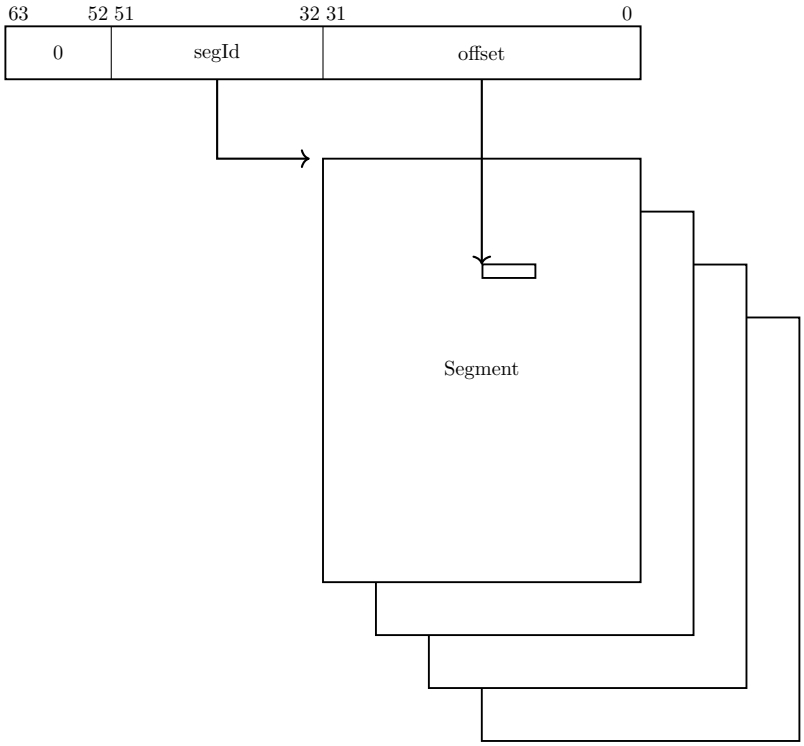


Figure 8: Virtual Memory Address Range

A virtual address is translated into a physical address. For translation purposes, the virtual address range can also be viewed as a set of virtual pages. The architected page size is 16 KB, with a 38-bit virtual page number and a 14-bit page offset.

Address Space Layout

Twin-64 architecture features a 36-bit physical memory address range which allows a maximum physical memory of up to 64 GBytes. The following figure depicts the overall memory layout.

Segment IDs 0 to 15 represent the physical address space, with a maximum size of 64 GB. These segments are directly mapped to physical addresses, without TLB translation or access-rights

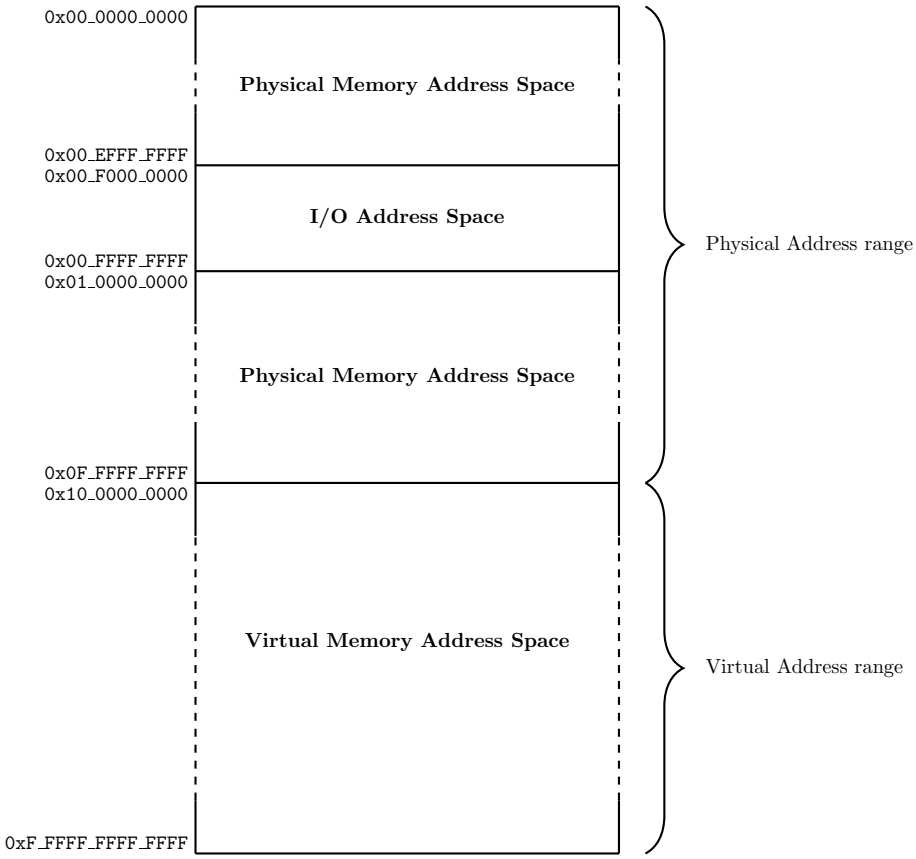


Figure 9: Memory Address Ranges

checking. Only privileged-mode code can access these segments. Control Register 0 contains a field that encodes the actual memory size installed in the system.

Twin-64 uses a memory-mapped I/O architecture. The I/O address space occupies the top 256 MB of the first segment. I/O modules allocate their registers and storage within this range, and access to this area is uncached.

The remaining address space is dedicated to virtual memory. Virtual storage is allocated dynamically, and the TLB provides the necessary translation to physical addresses while enforcing access rights.

Addressing and Access Control

The CPU generates 52-bit virtual addresses. Each virtual memory access must be translated and checked for valid access rights. This chapter describes the address translation process, as well as the access-rights and protection mechanisms.

Address Translation

Each virtual memory access generated by the processor must be translated into a physical address. The translation process maps a virtual page to a physical page. Virtual pages in the first 16 segments bypass the TLB, meaning that the virtual address is identical to the physical address.

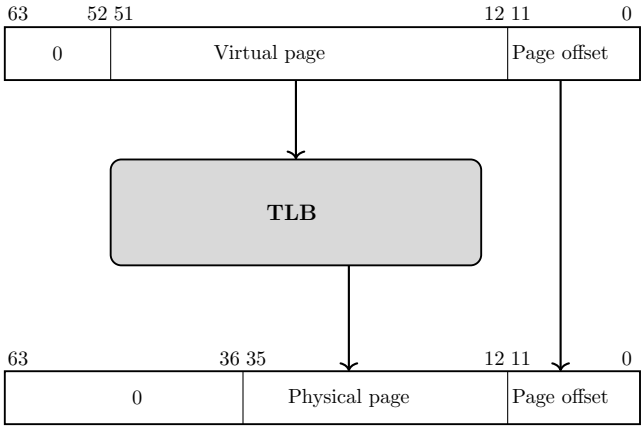


Figure 10: Address translation

Access Control

Twin-64 implements a protection model along two dimensions. The first dimension is **access control** and **privilege level**. Each page is associated with a page type: read-only, read-write, execute, or gateway. The page type is encoded in the AccType field, where 0 represents read-only access, 1 represents read-write access, 2 represents execute access, and 3 represents gateway access.

Each memory access is checked against the allowed access type defined in the page descriptor. There are two privilege levels: user mode and supervisor mode. The combination of access type and privilege level forms the access control information, which is verified for every instruction and data memory access.

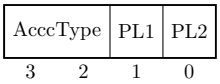


Figure 11: Access control information field

For read access, the privilege level must be at least as high as the PL1 field; for write access, it must be at least as high as PL2. Exceptions are execute and gateway pages, where write access is allowed only for privileged code. For execute access, the privilege level must be at least PL1 and no higher than PL2. If the instructions privilege level falls outside these bounds, a privilege violation trap is raised. If the page type does not match the instructions access type, an access violation trap is raised. In both cases, the instruction is aborted, and a memory protection trap occurs.

Table 4: Access type checking

Type	**read**	**write**	**execute**
`0 - read-only`	`PL <= PL1`	`not allowed`	`not allowed`
`1 - read-write`	`PL <= PL1`	`PL <= PL2`	`not allowed`
`2 - execute`	`PL <= PL1`	`PL == 0`	`PL2 <= PL <= PL1`
`3 - gateway`	`PL <= PL1`	`PL == 0`	`PL2 <= PL <= PL1`

Access Protection

The second dimension of protection is the **protection ID**. Twin-64 allows the processor to store up to eight protection ID values in the control registers `CR4` through `CR7`, which are accessible to the currently executing process. The protection ID is identical to a segment ID. For each access to a segment with the protection-check bit set, at least one of the protection ID registers must match the segment ID; otherwise, a protection violation trap is raised.

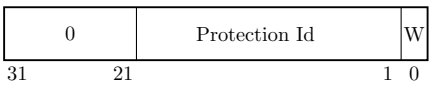


Figure 12: Protection Id

When protection ID checking is enabled, the eight protection identifiers are compared with the segment ID of the target address to validate an access. The rightmost bit of each protection ID register serves as a write-disable (W) bit. When the W bit is set, that protection ID cannot be used to grant write access. If the PSW P-bit is cleared, protection ID and write-disable checks are bypassed.

Protection ID checking is typically used to enforce controlled access to shared or private memory regions. A common example is the stack data segment, which is accessible at user privilege level. Since the CPU implements a global virtual memory address space, any task could potentially access the stack of another task. By enabling protection ID checking and loading the segment ID into one of the protection ID registers for the stack owner process, only the corresponding user task is allowed to access its stack data segment with a matching protection ID.

Note: Is the protection Id bit in the processor status word required? When we have the rule that privilege code can access anything, and user code will always be subject to protection Id checking, the bit is not needed.

Access Check Flow

The following figures summarizes the protection and access mode checking during address translation.

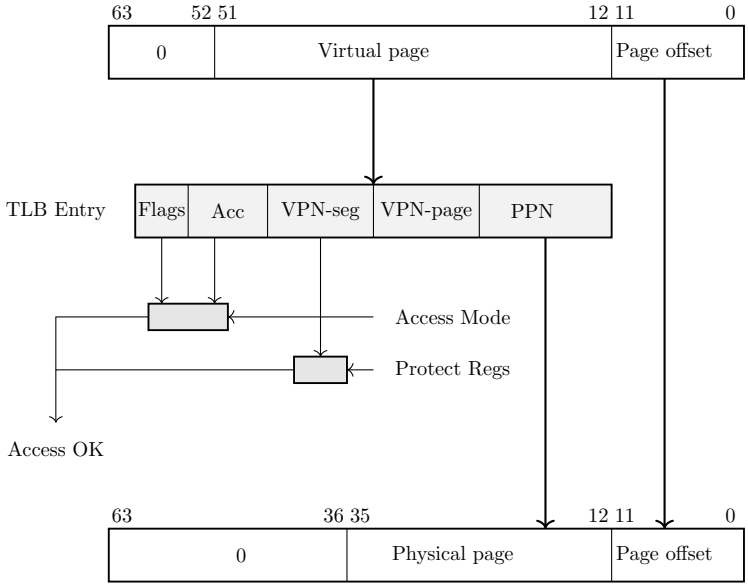


Figure 13: Address translation

For a successfully found TLB entry, the segment portion of the virtual address is compared against the set of protection Id registers. If there is no match, a protection Id trap is raised. The access mode of the instruction is compared against the access rights and TLB flags of the TLB entry. If the access mode does not match an access rights trap or a privilege violation trap is raised.

Control Flow

In Twin-64, control normally proceeds to the next sequential instruction in memory unless explicitly changed by a branch or interrupted by an exception. From the programmers perspective, instructions execute in program order. Internally, however, the hardware may employ techniques such as out-of-order execution. This chapter presents the logical execution model, describing how branching and interruptions alter control flow.

Unconditional Branches

Unconditional branch instructions always redirect control flow and may optionally save the return address in a general-purpose register. The branch target address can be computed in two ways:

- **Instruction-relative branches:** The target is obtained by adding an offset to the address of the current instruction.
- **Segment-relative branches:** The target is computed relative to the start of the segment (address zero of the segment).

Both methods use 32-bit unsigned arithmetic with wrap-around on overflow. For example, if the current instruction resides at `0xFFFF_FFFC` and the branch offset is `0x10`, the calculation is:

$$\texttt{0xFFFF\_FFFC} + \texttt{0x10} = \texttt{0x1\_0000\_000C}$$

Since the addition is performed modulo 2^{32}, the high-order carry is discarded. The effective branch target is therefore `0x0000_000C`.

Conditional Branches

Conditional branches combine a comparison with an instruction-relative branch. If the specified condition is true, control transfers to the branch target; otherwise, execution continues sequentially. Conditional branches are always instruction-relative.

Linkage

Certain branch instructions support procedure calls by saving the return address (the instruction immediately following the branch) into a general-purpose register. The linkage mechanism applies to both intra-segment and inter-segment branches. The saved return address consists of both the segment identifier and the instruction offset.

Inter-segment Branches

Unconditional inter-segment branches are branches to another segment. The offset is computed from the offset portion in the address specified plus the immediate value encoded in the `BE` instruction.

Privilege Level Changes

Branch instructions may alter the current privilege level. Execution is only permitted if the current privilege level satisfies the requirements of the code or data being accessed. Traditionally, privilege changes are handled through traps into the operating system kernel. While secure, this approach incurs significant overhead due to pipeline flushes and handler setup. Twin-64 provides a more efficient gateway mechanism. An unconditional branch with the `.G` gateway option may enter a gateway page, raising the privilege level. The processor then executes at the privilege level specified by the gateway page, with the PSW privilege bit updated accordingly. When returning to code in a lower-privilege page, the CPU automatically lowers the current privilege level. Each instruction fetch checks the privilege level of the code page against the PSW `P`-bit:

- If the page requires higher privilege than the PSW indicates, a privilege violation trap occurs.
- If the page requires lower privilege, execution privilege level is automatically demoted.
- If the G option on the branch instruction `B`is set, the execution privilege is set to the privilege level of the gateway page where the branch instruction resides.

Traps Associated with Branches

Branch instructions may trigger traps depending on the PSW state. For example, if the PSW `T`-bit is set and a branch is taken, a *taken-branch trap* is raised. This feature is primarily intended for debugging.

Interruptions

Twin-64 features a simple but powerful interrupt system. Interruptions in Twin-64 are divided into four classesfaults, traps, interrupts, and checksthat may occur during instruction execution.

- **Fault** - The current instruction cannot be carried out due to a system problem, such as the absence of a page from main memory. After the system problem has been corrected, the faulting instruction should execute as intended. Faults are synchronous with respect to the instruction stream.
- **Trap** - There are two kind of traps. The types are traps raised because the function requested by the current instruction cannot or should not be carried out. A good example is the arithmetic signed overflow trap. Such an instruction is normally not re-executed. The second type of traps are raised so that the system can intervene before the or after the instruction is executed. An example for this type of trap is the debugging breakpoint trap. Traps are synchronous with respect to the instruction stream.
- **Interrupt** - An external entity such as an I/O module requires attention. Interrupts are asynchronous with respect to the instruction stream.
- **Check** - The processor has detected an internal fault or malfunction. Checks can be either synchronous or asynchronous with respect to the instruction stream.

All four classes of interruptions are handled in the same way.

Interrupt Handling

Handling of an interrupt is implemented as a fast context switch. When an interruption occurs, the hardware saves the PSW at the time of the interrupt to the IPSW control register. PSW in effect at the time of the interruption is saved in the IPSW. Depending on the interrupt type, this can be the current instruction of the next instruction. All status bits, which are part of the PSW, are saved too. The hardware will then set the PSW status bits as follows. The M bit is set if the interruption is a high priority machine check. All other bits are set to zero. Depending on the trap type, additional data is saved to the interrupt argument control registers IARG0 and IARG1.

The control register `IVA` holds the virtual address of the interrupt vector table. Execution of the interrupt handler begins at the address computed using the trap number:

$$\texttt{IVA} + (32 * \texttt{trap number})$$

Each entry consists of 8 instructions that can be executed. It is the responsibility of interruption handlers to unmask external interrupts as soon as possible, so as to minimize interrupt latency. The interruptions are categorized into four groups.

Example. Suppose the interrupt vector table base address is:

$\texttt{IVA} = \texttt{0x0000\_1000}$

Now, if a **trap number 5** (Illegal Instruction) occurs, the handler entry point is calculated as:

$\texttt{0x0000\_1000} + (32 * 5) = \texttt{0x0000\_10A0}$

Execution of the interrupt handler starts at `0x0000_10A0`.

Interrupt Vector Table

Table 5: Trap Table

Trap	Name	Instruction Adr	Arg0	Arg1
1	Machine Check	current instruction	check type	-
2	Power Failure	current instruction	-	-
3	Recovery Counter	current instruction	-	-
4	External Interrupt	next instruction	-	-
5	Illegal instruction	current instruction	instruction	-
6	Privileged operation	current instruction	instruction	-
7	Privileged register	current instruction	instruction	-
8	Integer Overflow	current instruction	instruction	-
9	Instruction TLB miss	current instruction	-	-
10	Non-access Instruction TLB miss	current instruction	-	-
11	Instruction memory protection	current instruction	-	-
12	Data TLB miss	current instruction	instruction	target address
13	Non-access Data TLB miss	current instruction	instruction	target address
14	Data memory access rights	current instruction	instruction	target address
15	Data memory protection	current instruction	instruction	target address
16	Page reference	current instruction	instruction	target address
17	Alignment	current instruction	instruction	target address
18	Break Instruction	current instruction	instruction	-
19	Taken branch	next instruction	instruction	-

Part III

Instruction Set

Instruction Set

This part of the book presents the Twin-64 instruction set. Following a typical RISC design, it features a fixed-length format with a small number of regular instruction types. The total number of instructions is relatively small; however, many instructions encode execution options that make them highly versatile. Great care is taken to ensure that these options do not significantly complicate the pipeline or increase the overall data path length. Instead of supporting a wide variety of addressing modes, as commonly found in CISC-style CPUs, Twin-64 provides two virtual memory addressing modes based on a simple base-offset calculation. No indirection or addressing mode that requires reading a memory operand to compute an address is part of the architecture.

The instruction set is divided into five groups.

- **Memory reference instructions:** Load and store operations for virtual and physical memory.
- **Immediate instructions:** Building immediate values up to 64 bits.
- **Control flow instructions:** Conditional and unconditional branches.
- **Computational instructions:** Arithmetic, Boolean, and bit operations such as extract and deposit.
- **System control instructions:** Managing hardware resources such as caches and TLBs, register movement, traps, and interrupt handling.

Arithmetic Instructions

The arithmetic instructions operate on two signed 64-bit operands and store the result in the target register. There are two instruction layouts: signed immediate S15 and register-based. The immediate operand instruction format adds the sign-extended 15-bit value to `RegB`. The register instruction format uses two registers as input operands, performs the operation, and stores the result in a third register.

The `ADD` and `SUB` instructions perform signed 64-bit integer arithmetic, with numeric overflow triggering an overflow trap. There are no operations on unsigned quantities. The `SHLxA` and `SHRxA` instructions perform an arithmetic left or right shift on the input operand and add another operand to the shifted result, storing the final value in the target register. The `CMP` instruction compares two registers for equality, inequality, less than, and less than or equal conditions, storing the result in the target register.

Bit Operations Instructions

The bit operation instructions fall into two groups. The `AND`, `OR`, and `XOR` instructions perform the logical operations indicated by their names. Like the arithmetic instructions, they have both an immediate instruction format and a register instruction format.

The second group implements bit manipulation operations. The `EXTR` instruction extracts a bit field from a register and places it in another register; optionally, the extracted field can be sign-extended. The `DEP` instruction deposits a bit field into a target register. The `DSR` instruction concatenates two general-purpose registers, shifts the resulting 128-bit quantity to the right, and stores the lower 64 bits in the target register.

Immediate Instructions

The `LDI` and `ADDIL` instructions are used for forming large constants. With a 32-bit instruction word, it is not possible to encode even a full 32-bit constant in a single instruction. The `LDI` instruction includes a qualifier to load a constant value at a defined position in the target register. The `opt` field allows selection of the specific fields in the target register where the constant value is stored. Using a short sequence of instructions, any 64-bit value can be loaded.

The `ADDIL` instruction adds the upper portion of a 32-bit immediate value to the lower 32-bit half of a general register. This instruction is primarily used for address arithmetic, enabling access to any location within a segment. The `LDO` instruction loads an offset into a general register. The address is formed by adding the signed immediate value to a base register. Address arithmetic is performed as unsigned 32-bit arithmetic, wrapping around within the 32-bit range.

Memory Reference Instructions

Twin-64 is a registermemory architecture. It supports the common load/store instructions as well as instructions that combine an operation with a memory reference. Memory reference instructions feature two addressing modes. The first is the base register plus offset mode, and the second is the base register plus register index mode.

The indexed instruction format loads the content of the memory address formed by adding the scaled 13-bit immediate field to the base register `RegB`, and adds the fetched data value to `RegA`. The `dw` field specifies the data width: a `dw` value of 0 loads an 8-bit value, 1 loads a 16-bit value, 2 loads a 32-bit value, and 3 loads a 64-bit value. In all cases, the fetched value is sign-extended to 64 bits. Additionally, the `dw` field scales the offset by left-shifting the immediate field according to the data width. For example, a `dw` value of 3 loads a double word from an address formed by shifting the 13-bit offset by three bits before adding it to the base register.

The register-indexed instruction format loads the content of the memory address formed by adding `RegX` to the base register `RegB`. The `dw` field specifies the data width, as in the indexed instruction format. The offset in `RegX` is interpreted as a byte offset, independent of the `dw` value. In both instruction formats, the fetched data is sign-extended to 64 bits.

The LD and ST instructions are the standard load and store instructions. In addition to the two addressing modes described above, they also support base register modification. Depending on the sign of the offset, the base register is updated either before or after computing the effective address. See the instruction descriptions for more details.

The LDR and STC instructions provide the base support for a pipeline-friendly method for semaphore operations. They only support the indexed instruction mode with double-word access, and the base register modification option is not available.

The ADD, SUB, AND, OR, XOR, and CMP instructions perform the respective operations as described above, with one operand fetched from memory using the indexed addressing modes.

Control Flow Instructions

Control flow is implemented through a set of branch instructions, which can be classified into unconditional and conditional branches.

Unconditional branches fetch the next instruction address from the computed branch target. The B and BR instructions perform an instruction-relative branch. Both instructions can optionally store the return address in a general register.

The BV instruction performs a vectored branch. The content of a general register is added to a base register to form the branch target.

The BB, CBR, MBR, and ABR instructions implement conditional branching. If the specified condition is met, a branch to the target address is performed. The target address is always a local address, with the offset being instruction-address relative. Conditional branches adopt a static prediction model where forward branches are assumed not taken, while backward branches are assumed taken.

All branch address arithmetic is performed as 32-bit unsigned arithmetic with wraparound. Adding branch offsets will not overflow into the segment Id portion of the address.

System Instructions

The final instruction group is the **special/system** instruction group. The system control instructions manage CPU resources such as the TLB, cache, and other hardware components. Most instructions in this group require the processor to run in privileged mode.

The MFCR and MTCR instructions are used to transfer data to and from a control register. Access to some control registers requires privileged mode. The MFIA instruction accesses the current instruction address offset, which is important for position-independent code. The RSM and SSM instructions allow setting or clearing an individual accessible bit in the status portion of the processor state.

The LDPA, PRBR, and PRBW instructions are used to obtain information about the virtual memory system. The LDPA instruction returns the physical address of the virtual page, if present in memory. The PRBx instructions test whether the desired access mode to an address is allowed.

The translation lookaside buffers and caches are managed by the `ITLB`, `PTLB`, `FCA`, and `PCA` instructions. The `ITLB` and `PTLB` instructions insert or remove translations from the TLB. The `FCA` and `PCA` instructions manage the instruction and data caches, providing options to flush or purge a cache line. There is no instruction to insert data into a cache, as cache insertion is fully controlled by hardware.

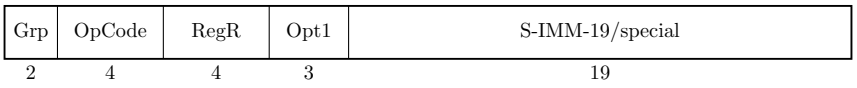
The `RFI` instruction restores the processor state from the control registers. Optionally, general registers stored in reserved control registers will also be restored.

The `DIAG` instruction issues hardware-specific implementation commands. An example is the memory barrier command, which ensures that all memory access operations complete before subsequent instructions execute.

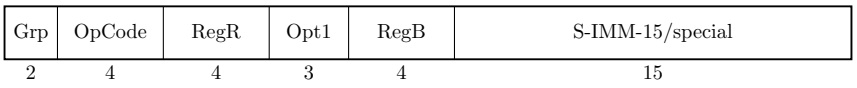
The `TRAP` instruction raises a software exception and is typically used to implement a debugging breakpoint.

Instruction Formats

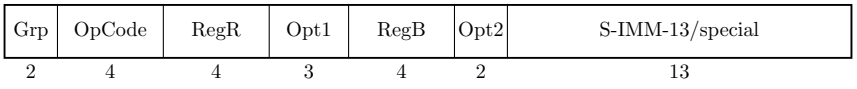
Twin-64 employs a small number of main instruction formats, which are classified according to the number of register operands and the length of the immediate value field. The **signed immediate S19 / special (S19)** format provides one register operand and a 19-bit signed immediate field. This format is typically used for branch instructions.



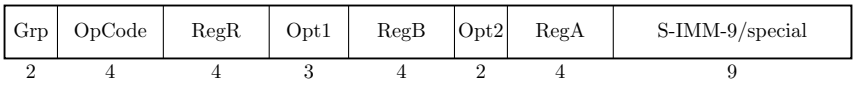
The **signed immediate S15 / special (S15)** instruction format provides two register operands and a 15-bit signed immediate field. This format is used both by conditional branch instructions and by instructions that operate on a register and an immediate value.



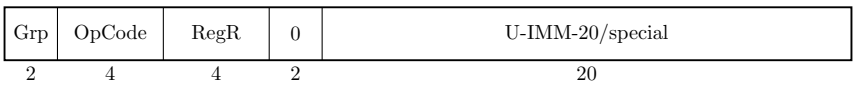
The **signed immediate S13 / special (S13)** instruction format provides two registers, an additional option field, and a 13-bit signed immediate field. Some instructions use the S-IMM-13 field for special encodings instead of a signed value. This format is used by memory reference instructions, where the address is formed by a base register plus a 13-bit signed offset.



The **register / signed immediate S9 / special (S9)** instruction format provides three registers, an additional option field, and a 9-bit signed immediate field. Some instructions use the S-IMM-9 field for special encodings instead of a signed value. This instruction format is used for all computational instructions that require three registers.



The **unsigned immediate U20 (U20)** format is used by the immediate instruction group to provide a 20-bit unsigned value. This value is then placed at specific positions in the target register `Reg R`.



In the instruction descriptions, each instruction format is referred to by its abbreviation (such as S9, S15, and so on). These formats specify the arrangement of operands, control bits, and optional immediate values within the instruction word. For some instructions, the immediate field directly encodes a constant, while in others the field is reserved or implicitly set according to the instructions encoding rules.

Instruction Set Descriptions

This chapter provides a detailed description of the Twin-64 instruction set. Each instruction entry includes:

- The instruction word layout.
- A concise description of the instruction.
- A high-level pseudo code representation of its operation.

Instruction operations are expressed using C-style pseudo code. The routines and conventions used are listed in the appendix. Registers are referenced by class and index:

- `GR[5]` general register 5
- `CR[1]` control register 1
- Some registers have dedicated names, e.g., the shift amount control register is `SAR`.

Subfields of an instruction are accessed using a dot and the field name in square brackets. For example:

- The interrupt enable bit in the PSW register: `PSW.[I]`.

Instruction options are specified in assembler syntax by a dot followed by one or more characters, each representing a defined option. For example:

- `AND.N ...` the `AND` instruction with the negate result option.
- The order of option characters does not matter; all defined options are listed in the relevant character set.

Fields marked as reserved are intended for future enhancements and should always be set to zero. Any other value in a reserved field results in undefined behavior.

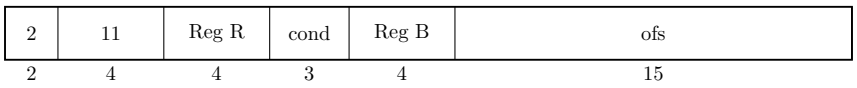
ABR - Add and branch

Syntax

```
ABR.<cond> RegR, RegB, target
```

Format

The ABR instruction uses the signed immediate S15 format:



Description

The `ABR` instruction adds `RegB` to `RegR` and stores the result in `RegR`. If the condition specified in the `cond` field is satisfied, execution continues at the current instruction plus the offset; otherwise, execution proceeds with the next instruction.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	RegR == 0
1	LT	RegR < 0
2	GT	RegR > 0
3	EV	RegR.[0] == 0
4	NE	RegR != 0
5	GE	RegR >= 0
6	LE	RegR <= 0
7	OD	RegR.[0] != 0

Operation

```
RegR <- RegR + RegB;
if ( cond( RegR )) PSW.[IA] <- PSW.[IA] + ofs;
else               PSW.[IA] <- PSW.[IA] + 4;
```

Exceptions

Overflow trap.

Notes

The `CBR` instruction extends the `CMP` functionality by integrating a conditional branch. With a mode field of three bits available for the `cond` field, all six standard relational conditions (`EQ`, `NE`, `LT`, `LE`, `GT`, `GE`) are directly encoded, avoiding the need for operand swapping or result inversion.

See also the instruction notes on comparison and branch Condition encoding in the appendix.

ADD Integer Addition

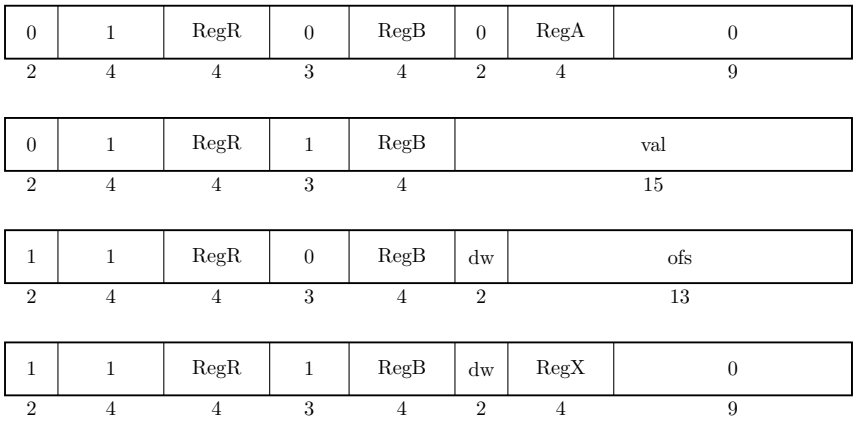
Syntax

```
ADD RegR, RegB, RegA
ADD RegR, RegB, val

ADD[.D/W/H/B] RegR, ofs ( RegB )
ADD[.D/W/H/B] RegR, RegX ( RegB )
```

Formats

The ADD instruction uses the following instruction formats.



Description

The `ADD` instruction adds two signed 64-bit operands and stores the result in the target register `RegR`. This instruction supports the immediate, register, and both memory-reference formats. An arithmetic overflow triggers a numeric overflow trap, and indexed instruction formats may cause a memory-reference related trap.

Operation

```
RegR <- RegB + RegA;                                    (S9 )
RegR <- RegB + signExt( val );                          (S15)
RegR <- RegR + memLoad( RegB + signExt( ofs << dw ));   (S13)
RegR <- RegR + memLoad( RegB + RegX );                  (S9 )
```

Exceptions

Integer Overflow Trap
Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

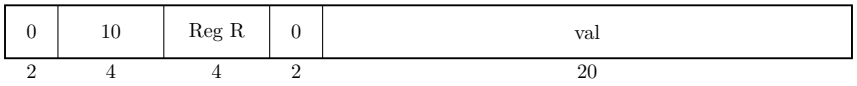
ADDIL - Add immediate left

Syntax

```
ADDIL RegR, val
```

Format

The ADDIL instruction uses the uses the following instruction formats.



Description

The `ADDIL` instruction adds an immediate value to a general register using 32-bit unsigned arithmetic. The 20-bit immediate value is shifted left by 12 bits (padded with zeros on the right) before being added to `RegR`. The resulting value is stored in the general register `GR1`. Any overflow that occurs during the addition is ignored.

Operation

```
RegR <- RegR + ( val << 12 );
```

Exceptions

None.

Notes

None.

AND - Boolean Operation

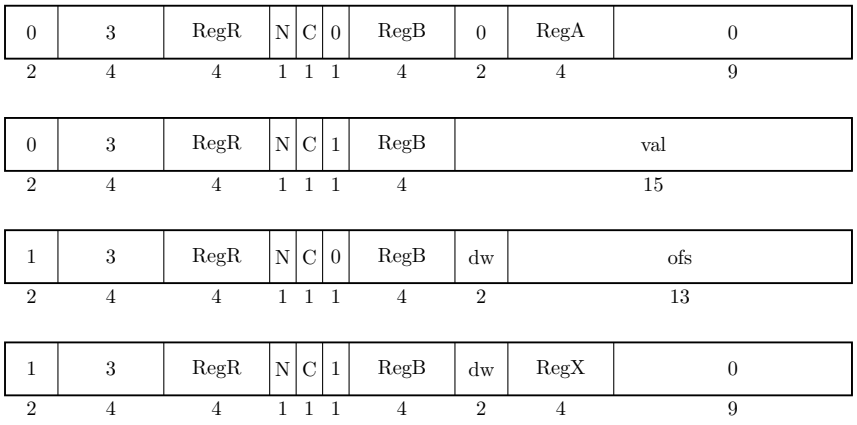
Syntax

```
AND[.C/N] RegR, RegB, RegA
AND[.C/N] RegR, RegB, val

AND[.C/N/D/W/H/B] RegR, ofs( RegB )
AND[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The AND instruction uses uses the following instruction formats.



Description

The `AND` instruction performs a bitwise AND operation on two 64-bit operands and stores the result in the target register `RegR`. The second operand, `RegB` or the loaded content, can be negated by setting the "C" bit, and the result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The Boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-reference related trap.

Operation

```
tmp <- RegA;                    (register format)
tmp <- immOperand( instr );     (immediate format)
tmp <- memOperand( instr );     (indexed formats)

if ( instr.[C] ) tmp <- not( tmp );
RegR <- RegB & tmp;
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

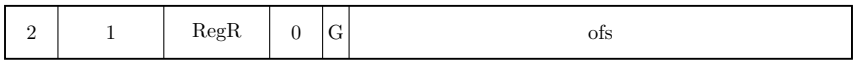
B Branch Unconditional

Syntax

```
B[.G] target [, RegR ]
```

Format

The `B` instruction uses the singed immediate S19 format.



Description

The `B` instruction performs an unconditional instruction-addressrelative branch. The target address is computed by sign-extending the immediate offset, shifting it left by two bits, and adding the result to the current instruction address. The return link is stored in `RegR`.

The optional `.G` form enables the *gateway branch* option. In this case, the processors privilege level may be raised to the privilege level stored in the TLB entry of the page from which the gateway instruction is fetched. The change is only performed if it results in a higher privilege level; otherwise, the current privilege level remains unchanged. Execution then continues at the computed target address with the (possibly new) privilege level.

Operation

```
if ( RegR != 0 ) {

        RegR.[51:32]<- PSW.[IA-REG];
        RegR.[31:0]  <- addOfs32( PSW.[IA-OFS], 4 );
}

if ( instr.[G] ) {

    if ( TLB[PSW.[IA-OFS]].[X] > PSW.[X] )
        PSW.[X] <- TLB[PSW.[IA]].[X];
}

PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], ofs << 2 );
```

Exceptions

Instruction Privilege Violation Trap
Instruction TLB Miss Trap
Taken Branch Trap

Notes

None.

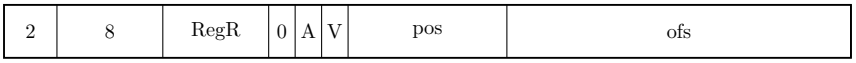
BB Branch on Bit

Syntax

```
BB[.T/F] ofs, pos
BB[.T/F] ofs, SAR
```

Format

The BB instruction uses the signed immediate S13 format.



Description

The BB instruction tests a bit in register RegR. The bit position can be specified directly by the pos field, or indirectly through the Shift Amount Register (SAR) if the A option is set.

If the V bit is set, the test succeeds when the selected bit is 1. If the V bit is clear, the test succeeds when the selected bit is 0.

When the condition is satisfied, execution branches to the target address, computed by sign-extending the offset, shifting it left by two bits, and adding it to the current instruction address. If the condition is not satisfied, execution continues with the next sequential instruction.

Operation

```
if ( instr.[A] )  pos <- SAR;
else              pos <- instr.[pos];

bVal <- testBit( RegR, pos );

cond <- ( instr.[V] ? ( bVal == 1 ) : ( bVal == 0 ));

if ( cond ) PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], ofs << 2 );
else        PSW.[IA-OFS] <- addOfs32( PSW.[IA-OFS], 4 );
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

Notes

None.

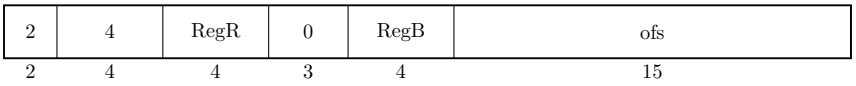
BE - Branch External

Syntax

```
BE [ ofs ] ( RegB ) [, RegR ]
```

Format

The `BE` instruction uses the signed immediate S15 format.



Description

The `BE` instruction performs an unconditional branch to another segment. `RegB` contains a target segment and offset base to which the sign extended offset shifted by two is added. The return link is stored in `RegR`.

Because `BE` is a segment base relative branch, branching to a page with a different privilege level is possible. Branching from a lower privilege level to a higher one triggers an instruction protection trap. Branching from a higher privilege to a lower privilege level updates the privilege level in the processor status register. If no change is required, the privilege level remains unchanged.

Operation

```
RegR <- addOfs32( PSW.[IA], 4 );
PSW.[IA] <- addOfs32( RegB, RegX << 2 );
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

Notes

None.

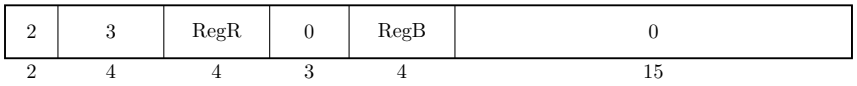
BR Branch Register

Syntax

```
BR RegB [, RegR ]
```

Format

The `BR` instruction uses the immediate-15 instruction format.



Description

The `BR` instruction performs an unconditional branch to a target address computed relative to the current instruction address segment (`IA`). The target address is formed by taking the contents of `RegB`, shifting it left by two bits, and adding it to the `IA` offset portion. The return link is stored in `RegR`.

Operation

```
RegR <- addOfs32( PSW.[IA], 4 );
PSW.[IA] <- addOfs32( PSW.[IA], RegB << 2 );
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

Notes

None.

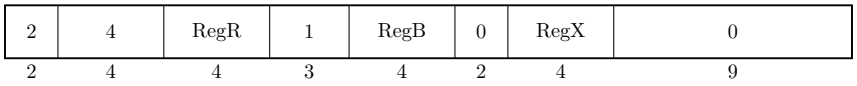
BV - Branch Vectored

Syntax

```
BV RegB [, RegR, RegX ]
```

Format

The `BV` instruction uses the register instruction format.



Description

The `BV` instruction performs an unconditional segment relative branch. The content of `RegX`is shifted by two and added to `RegB`, forming the target offset in the current segment. The return link is stored in `RegR`.

Because `BV` is a segment base relative branch, branching to a page with a different privilege level is possible. Branching from a lower privilege level to a higher one triggers an instruction protection trap. Branching from a higher privilege to a lower privilege level updates the privilege level in the processor status register. If no change is required, the privilege level remains unchanged.

Operation

```
RegR <- addOfs32( PSW.[IA], 4 );
PSW.[IA] <- addOfs32( RegB, RegX << 2 );
```

Exceptions

Taken Branch Trap
Instruction TLB Miss Trap

Notes

None.

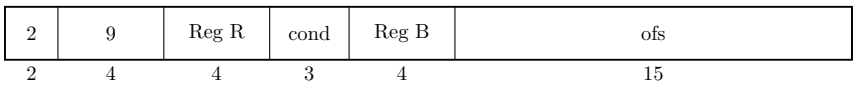
CBR - Compare and Branch

Syntax

```
CBR.<cond> RegR, RegB, target
```

Format

The CBR instruction uses the signed immediate S15 format:



Description

The `CBR` instruction compares the contents of `RegB` with `RegR` using the condition specified in the `cond` field. If the condition is satisfied, control is transferred to the instruction address computed by adding the sign-extended and word-aligned offset to the current instruction address. Otherwise, execution continues with the next sequential instruction. The comparison operation uses the same signed 64-bit comparator as the `CMP` instruction, and neither source register is modified.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	`RegR == RegB`
1	LT	`RegR < RegB`
2	GT	`RegR > RegB`
3	reserved	
4	NE	`RegR != RegB`
5	GE	`RegR >= RegB`
6	LE	`RegR <= RegB`
7	reserved	

Operation

```
if ( cond( RegR, RegB ))
   PSW.[IA] <- addOfs32( PSW.[IA], ( ofs << 2 ));
else
   PSW.[IA] <- addOfs32( PSW.[IA] + 4 );
```

Exceptions

Taken Branch Tap.
Instruction TLB Miss.

Notes

The `CBR` instruction extends the `CMP` functionality by integrating a conditional branch. With a mode field of three bits available for the `cond` field, all six standard relational conditions (`EQ`, `NE`, `LT`, `LE`, `GT`, `GE`) are directly encoded, avoiding the need for operand swapping or result inversion.

See also the instruction notes on comparison and branch Condition encoding in the appendix.

CMP - Compare

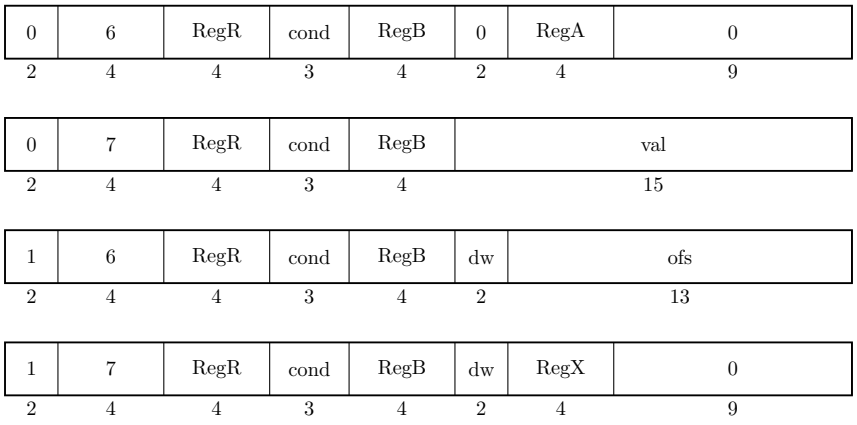
Syntax

```
CMP RegR, RegB, RegA
CMP RegR, RegB, val

CMP[.D/W/H/B] RegR, ofs ( RegB )
CMP[.D/W/H/B] RegR, RegX ( RegB )
```

Format

The CMP instruction uses the following instruction formats.



Description

The `CMP` instruction compares two signed 64-bit operands. In register mode, it compares `RegB` and `RegA`; in immediate mode, it compares `RegB` and the immediate value; and in memory-reference formats, it compares `RegR` with the memory operand. The operands are called `op1` and `op2`. The result of the comparison is stored in RegR. The instruction does not cause an arithmetic overflow but indexed instruction formats may cause a memory-referencerelated trap.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	`RegR == RegB`
1	LT	`RegR < RegB`
2	NE	`RegR != RegB`
3	GE	`RegR >= RegB`
4	LE	`RegR <= RegB`
5	reserved	
6	GT	`RegR > RegB`
7	reserved	

Operation

```
RegR <- compare( RegB, RegA );          (register format)
RegR <- compare( RegB, immOperand());   (immediate format)
RegR <- compare( RegR, memOperand());   (indexed formats)
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

See also the instruction notes on comparison and branch Condition encoding in the appendix.

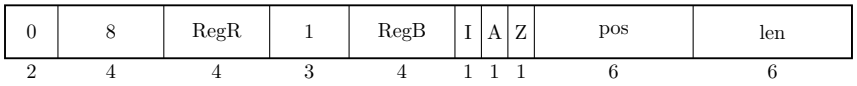
DEP - Deposit Bit Field

Syntax

```
DEP[.Z/I] RegR, RegB, pos, len
DEP[.Z/I] RegR, RegB, SAR, len
```

Format

The `DEP` instruction uses the special-13 instruction format.



Description

The `DEP` instruction deposits a bit field from general register `RegB` into `RegR` at the specified position.

The rightmost bit of the target field is specified by the `pos` instruction field, or by the Shift Amount Register (`SAR`) if the `A` bit is set. The length of the field is specified by the `len` instruction field.

If the `Z` bit is set, the target register `RegR` is cleared before depositing the field. If the `I` bit is set, the value to deposit is taken from the RegB field rather than a register in `RegB`.

Operation

```
if ( instr.[Z] ) RegR <- 0;

if ( instr.[A] ) pos <- SAR;
else             pos <- instr.[pos];

if ( instr.[I] )
    RegR <- depositImm( RegR, RegB, pos, instr.[len] );
else
    RegR <- depositField( RegR, RegB, pos, instr.[len] );
```

Exceptions

None

Notes

None

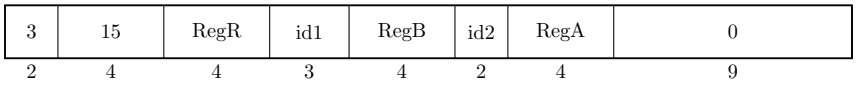
DIAG - Diagnose Instruction

Syntax

```
DIAG id, RegB, RegA
```

Format

The `DIAG` instruction uses the signed immediate S9 instruction format.



Description

The `DIAG` instruction provides a mechanism for diagnostic and system level operations.

The concatenated `id1` and `id2` fields specify the diagnostic function. The `RegB` and `RegA` fields may provide operands or data required by the diagnostic function. Any result is returned in `RegR`.

Operation

```
RegR <- diagOperation( concat( id1, id2 ), RegB, RegA );
```

Exceptions

Exceptions depend on diagnostic function and operands.

Notes

None

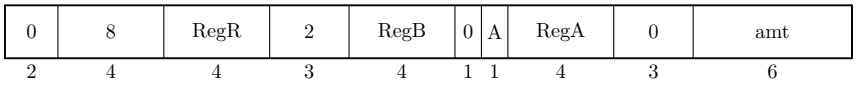
DSR Double Shift Right

Syntax

```
DSR RegR, RegB, RegA, amt
DSR RegR, RegB, RegA, SAR
```

Format

The `DSR` instruction uses the special-9 instruction format.



0	8	RegR	2	RegB	0	A	RegA	0	amt
2	4	4	3	4	1	1	4	3	6

Description

The `DSR` instruction concatenates the contents of registers `RegB` (left) and `RegA` (right) and performs a right shift operation. The lower 64 bits of the shifted result are stored in `RegR`.

The shift amount is specified by the `amt` instruction field, or by the Shift Amount Register (`SAR`) if the `A` bit is set.

Operation

```
if ( instr.[A] ) tmp <- SAR;
else             tmp <- instr.[amt];

RegR <- doubleShiftR( RegB, RegA, tmp );
```

Exceptions

None

Notes

None

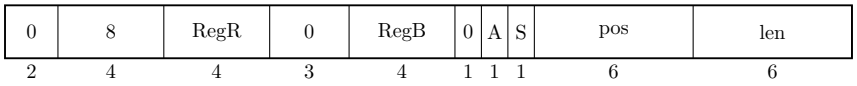
EXTR - Extract Bit Field

Syntax

```
EXTR[.S] RegR, RegB, pos, len
EXTR[.S] RegR, RegB, SAR, len
```

Format

The `EXTR` instruction uses the special-9 instruction format.



Description

The `EXTR` instruction extracts a bit field from general register `RegB`.

The rightmost bit of the field is specified by the `pos` instruction field. If the `A` bit is set, the position value is taken from the Shift Amount Register (`SAR`) instead of the instruction field.

The length of the field is specified by the `len` instruction field. The extracted bit field is stored right-justified in `RegR`. If the `S` bit is set, the extracted value is sign-extended.

Operation

```
if ( instr.[A] ) tmp <- SAR;
else             tmp <- instr.[pos];

RegR <- extractField( RegB, tmp, instr.[len] );

if ( instr.[S] ) RegR <- signExtend( RegR, instr.[len] );
```

Exceptions

None

Notes

None

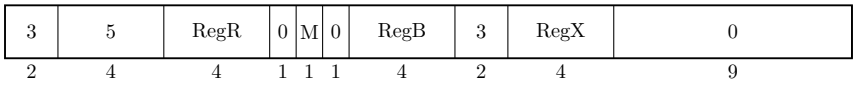
FxCA - flush from data cache

Syntax

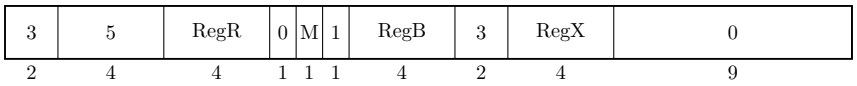
```
FICA RegR, [ RegX ] ( RegB )
FDCA RegR, [ RegX ] ( RegB )
```

Format

The `FICA` instruction uses the following instruction format.



The `FDCA` instruction uses the following instruction format.



Description

The `FCA` instruction flushes a cache line, selected by the effective address in `RegB`. If the M bit is set, the base register RegB is updated with the computed effective address. This is a privileged instruction.

Operation

```
RegR <- flushFromCache( addOfs32( RegB, RegX ));
```

Exceptions

Privilege violation trap.

Notes

We may enhance the instruction to have the "M" bit for supporting flush loops... tbd.

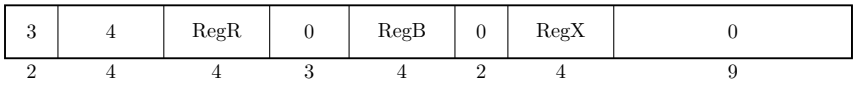
IxTLB Insert TLB Entry

Syntax

```
IITLB RegR, RegB, RegX
IDTLB RegR, RegB, RegX
```

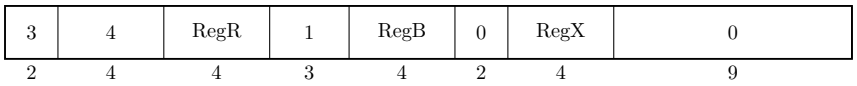
Format

The `IITLB` instruction uses the following instruction format.



3	4	RegR	0	RegB	0	RegX	0
2	4	4	3	4	2	4	9

The `IDTLB` instruction uses the following instruction format.



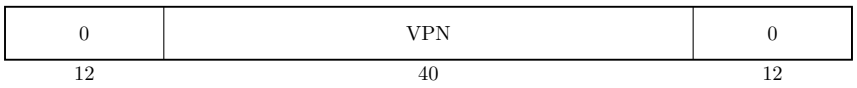
3	4	RegR	1	RegB	0	RegX	0
2	4	4	3	4	2	4	9

Description

The `ITLB` instruction inserts a new entry into the translation lookaside buffer (TLB). The effective address is stored in `RegB` and additional control information is provided by `RegX`. This is a privileged instruction.

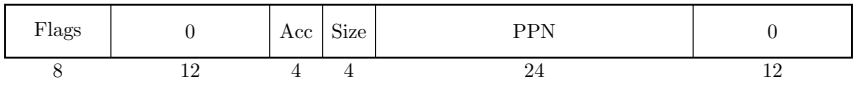
TLB Arguments

`RegB` and `RegA` contain the arguments passed to the TLB subsystem. `RegB` contains the virtual address. The address is always o a page boundary and in the case of address ranges marks the starting page.



0	VPN	0
12	40	12

`RegA` contains the flags and the physical address of the address range.



Flags	0	Acc	Size	PPN	0
8	12	4	4	24	12

Operation

```
RegR <- insertIntoTlb( RegB, RegA );
```

Exceptions

Privilege Violation Trap.

Notes

None.

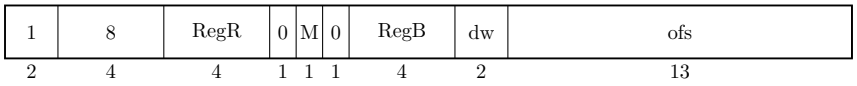
LD Load Register

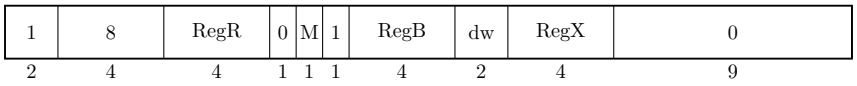
Syntax

```
LD[.D/W/H/B/M] RegR, SignedImmed13( RegB )
LD[.D/W/H/B/M] RegR, RegX( RegB )
```

Format

The `LD` instruction uses the indexed and register-indexed instruction formats.





Description

The `LD` instruction loads a data value from memory into a general-purpose register. Two formats are available: the immediate-offset format and the register-indexed format. In the immediate-offset format, the signed immediate offset is shifted right by the `dw` field of the instruction and added to `RegB` to form the effective address. In the register-indexed format, `RegX` is added to `RegB` to form the effective address.

If the M bit is set, the base register RegB is updated with the computed effective address. The effective address must be aligned to the data width specified. The value read from memory is sign-extended and placed in general register `RegR`.

Operation

```
RegR <- memLoad( memOperand( instr ));
```

Exceptions

- Data Alignment Trap
- Memory Access Violation Trap
- Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

The option to update the base register is often used in loops to fetch data from memory.

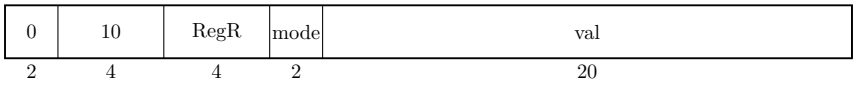
LDIL Load Immediate Left

Syntax

`LDIL[.L/.S/.U] RegR, val`

Format

The `LDIL` instruction uses the immediate-20 instruction format.



Description

The `LDIL` instruction deposits an immediate value into general register `RegR`. The optional suffix selects how the immediate field is positioned within the register. If no suffix is given, the default is `.L` mode (mode value = 1).

- `.L` (mode = 1) Deposits 20 bits of `val` into bits [31..12] of `RegR`.
- `.S` (mode = 2) Deposits 20 bits of `val` into bits [51..32] of `RegR`.
- `.U` (mode = 3) Deposits the lower 12 bits of `val` into bits [63..52] of `RegR`.

A mode value of zero is reserved and must not be used.

Operation

```
RegR <- ((val & 0xFFFFF) << 12)   // .L mode (1)
RegR <- ((val & 0xFFFFF) << 32)   // .S mode (2)
RegR <- ((val & 0xFFF)   << 52)   // .U mode (3)
```

Exceptions

None.

Notes

None.

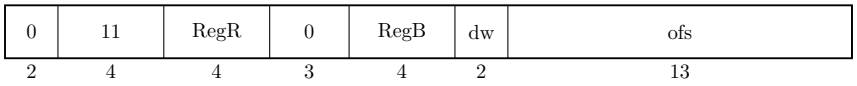
LDO Load Offset

Syntax

`LDO [.B/H/W/D] RegR, ofs(RegB)`

Format

The `LDO` instruction uses the immediate-15 instruction format.



Description

The `LDO` instruction loads a value from memory using an offset addressing mode. The `ofs` value is sign extended and shifted by the `dw` field. The result is calculated by adding this value to the contents of `RegB`. The loaded value is then stored in `RegR`.

Operation

```
RegR <- addOfs32(RegB, signExtend( ofs ) << 2 );
```

Exceptions

None.

Notes

None

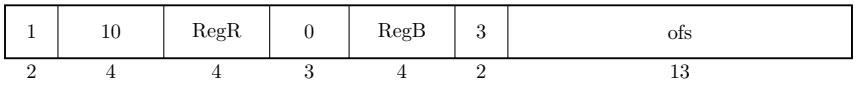
LDR Load Register Reserved

Syntax

```
LDR RegR, SignedImmed13(RegB)
```

Format

The `LDR` instruction uses the indexed instruction format.



Description

The `LDR` instruction loads a value from memory into a general-purpose register and marks the address as "reserved" for a subsequent conditional store. The instruction is only the first step of an atomic read-modify-write pair. The subsequent `STC` (store conditional) instruction will succeed only if the reservation is still valid.

The signed immediate offset is shifted right by 3 and added to `RegB` to form the effective address.

Operation

```
RegR <- memLoad( memOperand( instr ));
markAddressReserved(memOperand( instr ));
```

Exceptions

- Memory Access Violation Trap
- Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

This instruction is part of a "load-reserved / store-conditional" pair for atomic memory updates and often used for implementing locks or atomic counters. Reservation may be cleared by other stores to the same address by another processor.

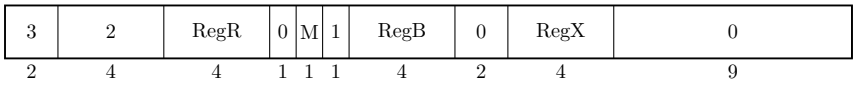
LPA Load Physical Address

Syntax

```
LPA[.D/W/H/B] RegR, RegX( RegB )
```

Format

The `LD` instruction uses the indexed and register-indexed instruction formats.



Description

The `LPA` (Load Physical Address) instruction computes the physical address corresponding to the given virtual address and loads it into `RegR`. The virtual address is formed by adding `RegX` to `RegB`. If there is no translation, a value of zero is returned.

The physical address, if found, is returned from information in the TLB. No memory access is performed. If there is a separate instruction and data TLB, the data TLB is used. For direct mapped segments the virtual address is the physical address.

If the M bit is set, the base register RegB is updated with the computed effective address.

Operation

```
RegR <- tanslateAdr( addOfs32( RegB + RegX ));
```

Exceptions

- Alignment Trap
- Privileged instruction trap.
- Non access data TLB trap.

Notes

None.

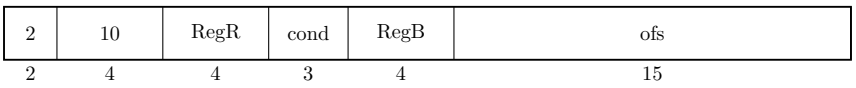
MBR - Move and Branch

Syntax

```
MBR.<cond> RegR, RegB, target
```

Format

The MBR instruction uses the following format:



Description

The `MBR` instruction copies the contents of `RegB` into `RegR`. If the condition specified in the `cond` field evaluates true for the new value in `RegR`, execution continues at the current instruction address plus the sign-extended offset. Otherwise, execution resumes with the next instruction.

Conditions

The `cond` field encodes the branch condition as follows:

Value	Mnemonic	Branch if
0	EQ	RegR == 0
1	LT	RegR < 0
2	GT	RegR > 0
3	EV	RegR.[0] == 0
4	NE	RegR != 0
5	GE	RegR >= 0
6	LE	RegR <= 0
7	OD	RegR.[0] != 0

Operation

```
RegR <- RegB;
if ( cond( RegR )) PSW.[IA] <- addOfs32( PSW.[IA], signExtend(
    ofs ) << 2 );
else               PSW.[IA] <- PSW.[IA] + 4;
```

Exceptions

None.

Notes

The `CBR` instruction extends the `CMP` functionality by integrating a conditional branch. With a mode field of three bits available for the `cond` field, all six standard relational conditions (`EQ`, `NE`, `LT`, `LE`, `GT`, `GE`) are directly encoded, avoiding the need for operand swapping or result inversion.

See also the instruction notes on comparison and branch Condition encoding in the appendix.

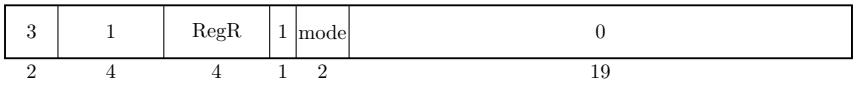
MFIA - Move from Instruction Address

Syntax

`MFIA [.U/S/L/A]RegR`

Format

The `MFIA` instruction uses the register instruction format.



Description

The `MFIA` instruction copies the current instruction address to register `RegR`.The mode field allows to select a portion of the instruction address.

- `.A` (mode = 0) copies IA to `RegR`. This is the default operation.
- `.L` (mode = 1) Deposits 20 bits of `val` into bits [31..12] of `RegR`.
- `.S` (mode = 2) Deposits 20 bits of `val` into bits [51..32] of `RegR`.
- `.U` (mode = 3) Deposits the lower 12 bits of `val` into bits [63..52] of `RegR`.

Operation

```
RegR <- PSW.[IA]                        // .A mode (0)
RegR <- ((PSW.[IA] & 0xFFFFF) << 12)    // .L mode (1)
RegR <- ((PSW.[IA] & 0xFFFFF) << 32)    // .S mode (2)
RegR <- ((PSW.[IA] & 0xFFF)   << 52)    // .U mode (3)
```

Exceptions

None.

Notes

It would be beneficial to have the possibility to just a subject of the instruction address. How about options to get the segment, the page number in the segment and the status bits. It is very similar to LDIL fields.

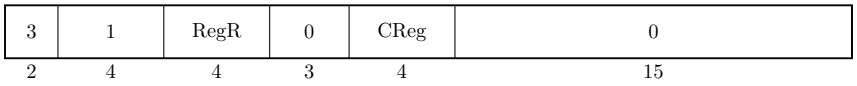
MFCR Move From Control Register

Syntax

```
MFCR RegR, CReg
```

Format

The `MFCR` instruction uses the control-register instruction format.



Description

The `MFCR` instruction copies the contents of control register `CRegB` to general register `RegR`. Some control registers are privileged. Attempting to read them in user mode results in a privilege exception.

Operation

```
RegR <- CReg;
```

Exceptions

- Privileged Operation Trap for some control registers.

Notes

None

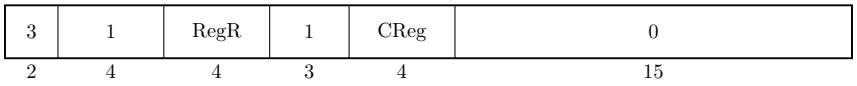
MTCR Move To Control Register

Syntax

```
MTCR CReg, RegR
```

Format

The `MTCR` instruction uses the control-register instruction format.



Description

The `MTCR` instruction copies the contents of general register `RegR` to control register `CReg`. Some control registers are privileged. Attempting to write them in user mode results in a privilege exception.

Operation

```
CReg <- RegR;
```

Exceptions

Privilege operation trap for some control registers.

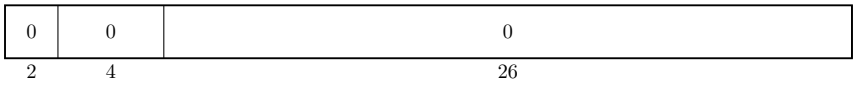
Notes

None

NOP No Operation

Syntax `NOP`

Format The `NOP` instruction uses no format.



Description The `NOP` (No Operation) instruction does nothing. It is typically used for timing, alignment, or to occupy a pipeline slot.

Operation

```
noOperation( );
```

Exceptions None.

Notes None.

OR Boolean Operation

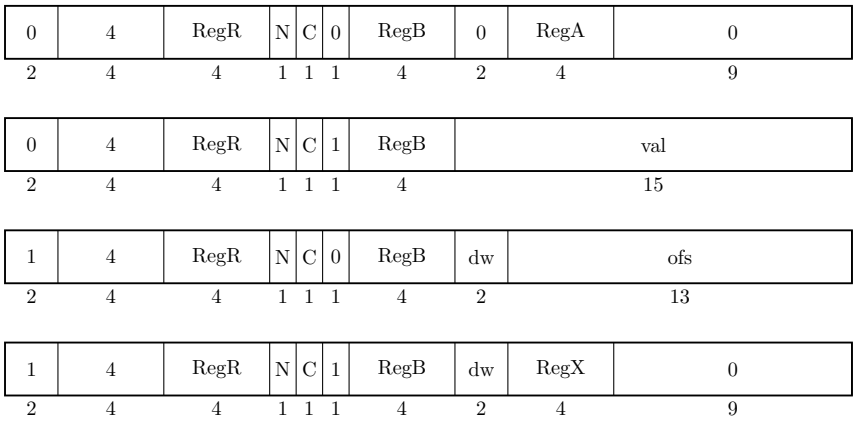
Syntax

```
OR[.C/N] RegR, RegB, RegA
OR[.C/N] RegR, RegB, val

OR[.C/N/D/W/H/B] RegR, ofs( RegB )
OR[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The OR instruction uses the register, the immediate-19, the indexed and the register indexed instruction formats.



Description

The `OR` instruction performs a bitwise OR operation on two 64-bit operands and stores the result in the target register `RegR`. The `AND` instruction performs a bitwise AND operation on two 64-bit operands and stores the result in the target register `RegR`. The second operand, `RegB` or the loaded content, can be negated by setting the "C" bit, and the result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The Boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-reference related trap.

Operation

```
tmp <- RegA;                  (register format)
tmp <- immOperand( instr );   (immediate format)
tmp <- memOperand( instr );   (indexed formats)

if ( instr.[C] ) tmp <- not( tmp );
RegR <- RegB | tmp;
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

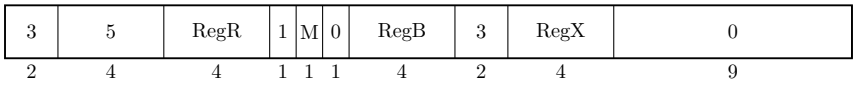
PxCA - Purge from Cache

Syntax

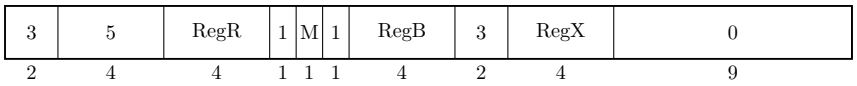
```
PICA RegR, RegB
PDCA RegR, RegB
```

Format

The `PICA` instruction uses the standard indexed instruction format.



The `PDCA` instruction uses the standard indexed instruction format.



Description

The `PICA` and `PDCA` instructions purge a cache line, selected by the effective address in `RegB`. This can be used by privileged code for cache management. Unlike `FDCA`, purge may also invalidate dirty lines without writing them back. The mode field indicates the cache type. A value of 0 refers to the instruction cache, a value of one refers to the data cache. A value of two refers to a unified cache. If the M bit is set, the base register RegB is updated with the computed effective address.

Operation

```
RegR <- purgeFromCache( addOfs32( RegB, RegX );
```

Exceptions

None.

Notes

Typically only allowed in supervisor mode.

// ??? also uses the index and register indexed formats ?

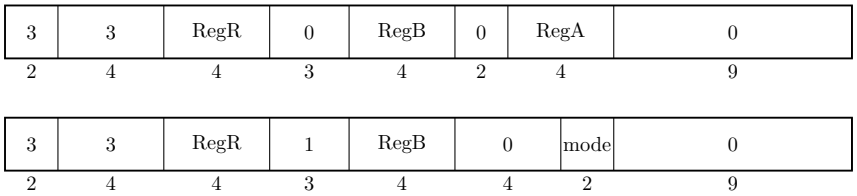
PRB Probe Virtual Address

Syntax

```
PRB RegR, RegB, RegA
PRB RegR, RegB, mode
```

Format

The `PRB` instruction uses both the register format and the special-9 format.



Description

The `PRB` instruction probes the Translation Lookaside Buffer (TLB) to determine whether the effective address in `RegB` has a valid mapping. The result of the probe is returned in `RegR`: `0` indicates that no mapping exists, while `1` indicates a valid mapping.

In the register format, the access mode is specified in `RegA`. In the special-9 format, the mode is encoded directly in the instruction.

The mode specifies the type of access being probed:

Value	Access type
0	`Read access`
1	`Write access`
2	`Execute access`
3	`Undefined`

Operation

```
RegR <- probeVirtualAddress(RegB, RegA.[62..63]);  // Reg
RegR <- probeVirtualAddress(RegB, mode);           // Mode
```

Exceptions

- Non-access TLB Trap
- Privileged Operation Trap

Notes

Typically used by the operating system for virtual memory management.

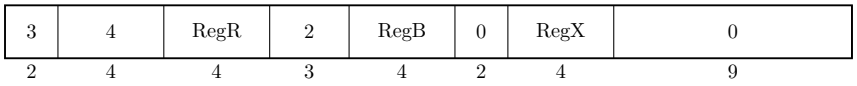
PxTLB Purge TLB Entry

Syntax

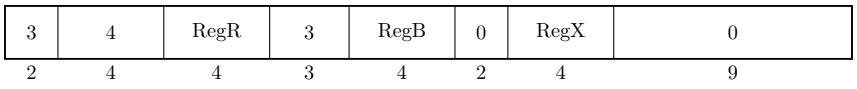
```
PITLB RegR, RegB, RegA
PDTLB RegR, RegB, RegA
```

Format

The `PITLB` instruction uses the following instruction format.



The `PDTLB` instruction uses the following instruction format.



Description

The `PITLB` and `PDTLB` instructions purge (removes) an entry from the translation lookaside buffer (TLB). The entry to remove is identified by the effective address formed from `RegB` and `RegX`. If the M bit is set, the base register RegB is updated with the computed effective address.This is privileged instruction.

Operation

```
RegR <- purgeFromTlb( RegB, RegA );
```

Exceptions

- Privileged operation trap.

Notes

None.

// ??? standard memory address formats ?

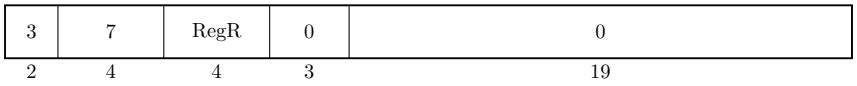
RFI Return from Interrupt

Syntax

```
RFI [ RegR ]
```

Format

The `RFI` instruction uses the branch instruction format.



Description

The `RFI` instruction restores processor state after an interrupt or exception handler has completed. Execution resumes at the program counter saved during the interrupt and the processor state (including privilege level, interrupt enable flags, and other PSR fields) is restored from the saved copy.

Operation

```
PSW <- IPSW;
```

Exceptions

Privileged operation trap.

Notes

None.

// ??? what is RegR used for ?

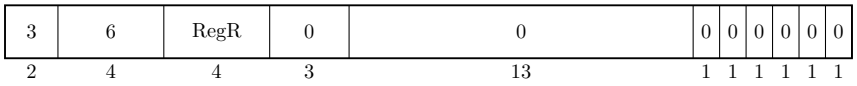
RSM Reset System Mask

Syntax

```
RSM RegR
```

Format

The `RSM` instruction uses the system instruction format.



Description

The `RSM` (Reset System Mask) instruction clears bits in the system mask register that are set in `RegR`.

Operation

```
SystemMask <- SystemMask & instr.[5:0];
```

Exceptions

- Privileged instruction trap.

Notes

None.

// ?? what is RegR used for ? return old mask ?

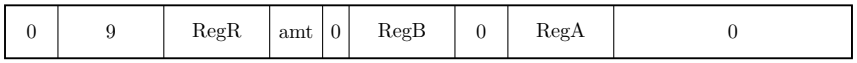
SHLxA Shift Left and Add

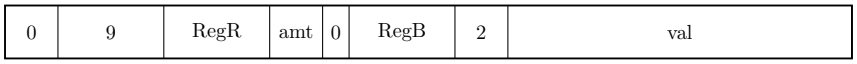
Syntax

```
SHLxA RegR, RegB, RegA
SHLxA RegR, RegB, SignedImmed13
```

Format

The `SHLxA` instruction supports both register and immediate formats.





Description

The `SHLxA` instruction shifts the contents of `RegB` left by the amount specified in the `amt` field. In register format, `RegA` is added to the shifted value. In immediate format, if the I bit is set, the sign-extended `val` field is added instead. The resulting value is stored in `RegR`.

Operation

```
RegR <- ( RegB << instr.[amt] ) + RegA;                (Register )
RegR <- ( RegB << instr.[amt] ) + immOperand( instr );(Immediate)
```

Exceptions

Integer Overflow Trap

Notes

None

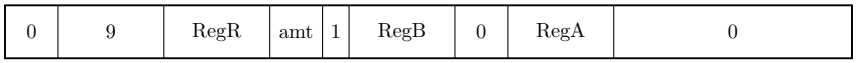
SHRxA - Shift Right and Add

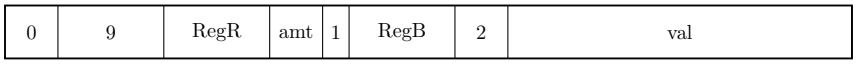
Syntax

```
SHRxA RegR, RegB, RegA
SHRxA RegR, RegB, val
```

Format

The `SHRxA` instruction supports both register and immediate formats.





Description

The `SHRxA` instruction shifts the contents of `RegB` right by the amount specified in the `amt` field. In register format, `RegA` is added to the shifted value. In immediate format, if the I bit is set, the sign-extended `val` field is added instead. The resulting value is stored in `RegR`.

Operation

```
RegR <- ( RegB >> instr.[amt] ) + RegA;                (Register )
RegR <- ( RegB >> instr.[amt] ) + immOperand( instr );(Immediate)
```

Exceptions

Integer Overflow Trap

Notes

None

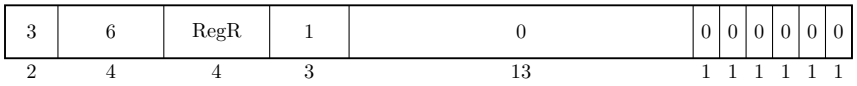
SSM Set System Mask

Syntax

```
SSM RegR, mask
```

Format

The `SSM` instruction uses the system instruction format.



Description

The `SSM` (Set System Mask) instruction updates the system mask register. The previous mask is returned in `RegR`.

Operation

```
SystemMask <- SystemMask | instr.[5:0];
```

Exceptions

- Privileged instruction trap.

Notes

None.

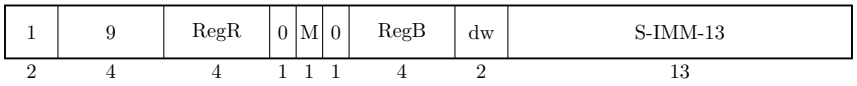
ST Store Register

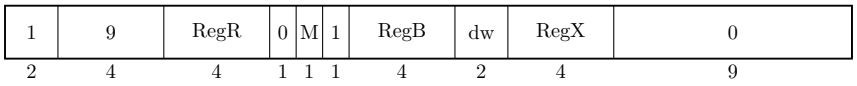
Syntax

```
ST[.D/W/H/B] RegR, SignedImmed13( RegB )
ST[.D/W/H/B] RegR, RegX( RegB )
```

Format

The `ST` instruction uses the indexed and register-indexed instruction formats.





Description

The `ST` instruction stores a value from a general-purpose register into memory.

In the immediate-offset format, the signed immediate offset is shifted right by the `dw` field of the instruction and added to `RegB` to form the effective address. In the register-indexed format, `RegX` is added to `RegB` to form the effective address.

If the M bit is set, the base register RegB is updated with the computed effective address. The store address must be aligned to the data width specified. The value stored to memory is stored as is in the data width specified.

Operation

```
memStore( memOperand( addOfs32( RegB, ofs )), RegR );
memStore( memOperand( addOfs32( RegB, RegX )), RegR );
```

Exceptions

- Data Alignment Trap
- Memory Access Violation Trap
- Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

The jury is still out if the M bit is not too expensive in hardware. We would need to read three registers. One for the value, and up to two for the index calculation.

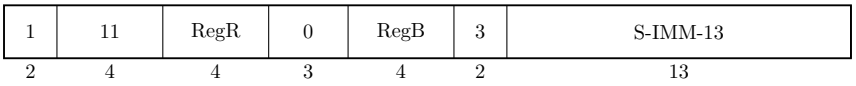
STC Store Register Conditional

Syntax

`STC RegR, SignedImmed13(RegB)`

Format

The `STC` instruction uses the indexed instruction format.



1	11	RegR	0	RegB	3	S-IMM-13
2	4	4	3	4	2	13

Description

The `STC` instruction stores a value from a general-purpose register to memory only if the memory address is still reserved by a prior `LDR` instruction. If the reservation is valid, the store succeeds and clears the reservation. If the reservation has been lost (e.g., another processor wrote to the address), the store fails and the memory is unchanged.

Operation

```
if ( isAddressReserved( memOperand( instr ))) {

    memStore( memOperand( instr ), RegR );
    clearReservation( memOperand( instr ));
    RegR <- 1;
}
else RegR <- 0;
```

Exceptions

- Data Alignment Trap
- Memory Access Violation Trap
- Memory Protection Violation Trap
- Data TLB Miss Trap

Notes

This instruction is the second part of a ”load-reserved / store-conditional” pair for atomic memory updates and commonly used for implementing spinlocks, mutexes, and atomic counters in multiprocessor systems.

SUB Integer Subtraction

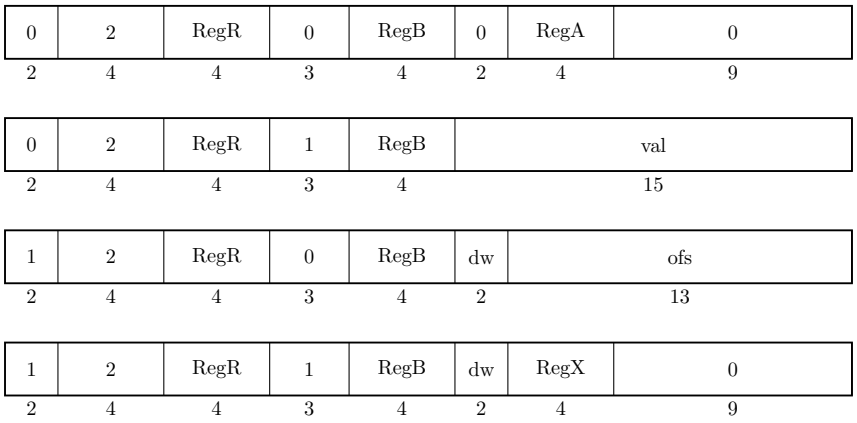
Syntax

```
SUB RegR, RegB, RegA
SUB RegR, RegB, val

SUB[.D/W/H/B] RegR, ofs ( RegB )
SUB[.D/W/H/B] RegR, RegX ( RegB )
```

Format

The SUB instruction uses the register, the immediate-15, the indexed and the register indexed instruction formats.



Description

The `SUB` instruction subtracts two signed 64-bit operands and stores the result in the target register `RegR`. This instruction supports the immediate, register, and both memory-reference formats. An arithmetic overflow triggers a numeric overflow trap, and indexed instruction formats may cause a memory-referencerelated trap.

Operation

```
RegR <- RegB - RegA;                ( register format )
RegR <- RegB - immOperand( instr ); (immediate format)
RegR <- RegR - memOperand( instr ); (indexed formats)
```

Exceptions

Integer Overflow Trap
Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

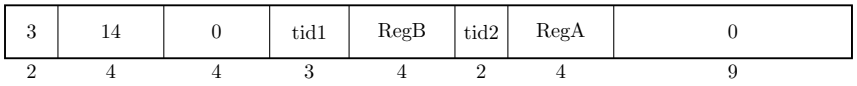
TRAP Trap Instruction

Syntax

```
TRAP tid, regB, regA
```

Format

The `TRAP` instruction uses the standard indexed instruction format.



3	14	0	tid1	RegB	tid2	RegA	0
2	4	4	3	4	2	4	9

Description

The `TRAP` instruction generates a synchronous trap to handle exceptional conditions. The `tid1` and `tid2` are concatenated to form the trap Id. The instruction may be used to invoke operating system routines or exception handlers.

Operation

```
raiseTrap( id, RegB, RegA );
```

Exceptions

The trap raised.

Notes

None

XOR Boolean Operation

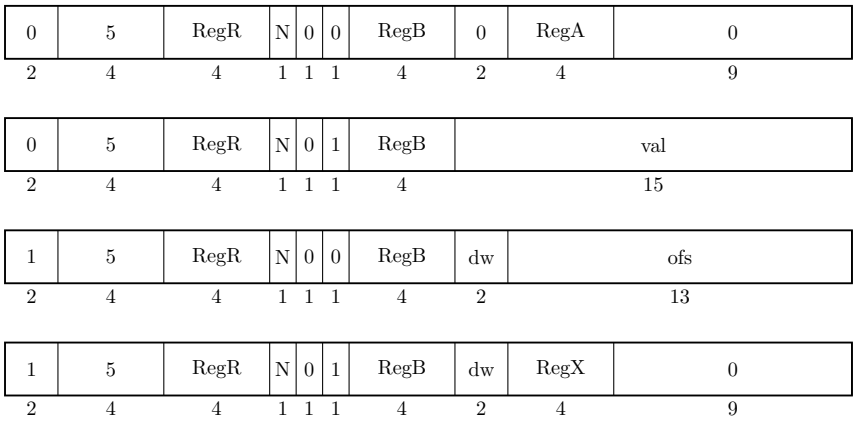
Syntax

```
XOR[.C/N] RegR, RegB, RegA
XOR[.C/N] RegR, RegB, val

XOR[.C/N/D/W/H/B] RegR, ofs( RegB )
XOR[.C/N/D/W/H/B] RegR, RegX ( RegB )
```

Format

The XOR instruction uses the register, the immediate-19, the indexed and the register indexed instruction formats.



Description

The `XOR` instruction performs a bitwise XOR operation on two 64-bit operands and stores the result in the target register `RegR`. The result can be negated by setting the "N" bit. This instruction supports the immediate, register, and both memory-reference formats. The Boolean operation itself does not trigger any traps, but the indexed formats may cause a memory-referencerelated trap.

Operation

```
RegR <- RegB ^ RegA;           (register format)
RegR <- RegB ^ immOperand();   (immediate format)
RegR <- RegR ^ memOperand();   (indexed formats)
if ( instr.[N] ) RegR <- not( RegR );
```

Exceptions

Data Alignment Trap
Memory Access Violation Trap
Memory Protection Violation Trap
Data TLB Miss Trap

Notes

None.

Synthetic Instructions

The Twin-64 instruction set provides a rich set of options for individual instructions. Setting a defined option bit in an instruction can enable additional functionality with minimal impact on the overall data path. Similarly, instructions that use general register zero as an argument can take advantage of predefined behaviors.

For better readability and convenience, an assembler can offer *synthetic instructions*, which are higher-level mnemonics generated from the base instruction set. These synthetic instructions simplify programming by providing a more natural or concise representation of commonly used instruction patterns.

Synthetic instructions are also useful when a single instruction cannot fully express the desired operation. For example, loading an immediate value larger than the instruction field allows requires an instruction sequence. An assembler can provide a generic "load immediate" synthetic instruction that automatically emits such an instruction sequence depending on the size of the value. Similarly, if the offset to a base register is too small to reach a desired data item, the assembler can transparently emit an additional 'ADDIL' instruction to compute the correct address.

The synthetic instruction set can be organized into several groups:

- **Immediate Operations:** yyy.
- **Register Operations:** yyy.
- **Arithmetic Operations:** yyy.
- **Shift and Rotate Operations:** yyy.
- **Memory Reference Operations:** yyy.

Immediate Operations

Table 6: Immediate synthetic instructions

OpCode	Sequence	Comment
LDI	...	...

Register Operations

Table 7: Immediate synthetic instructions

OpCode	Sequence	Comment
CLR	...	...
CPY	...	...
COM	...	...
NEG	...	...

Arithmetic Operations

Table 8: Immediate synthetic instructions

OpCode	Sequence	Comment
INC	...	...
DEC	...	...
EXT8	...	...
EXT16	...	...
EXT32	...	...

Shift and Rotate Operations

Table 9: Immediate synthetic instructions

OpCode	Sequence	Comment
ASR	...	...
LSR	...	...
ROR	...	...
ROL	...	...

Memory Reference Operations

Table 10: Immediate synthetic instructions

OpCode	Sequence	Comment
LOAD	...	...

Continued on next page

OpCode	Sequence	Comment
STOR	...	...

Part IV

I/O Subsystem

Input/Output

Concepts

Twin-64 features memory a mapped I/O architecture.

dedicated address space in physical memory

bus concept

module concept

modules have registers - load / store, however registers do not have to be 64 bit wide.

the system bus features up to 64 modules. that is: 64 HPA pages, need: 1Mb address range.

a module listens to three address ranges

- hard physical address range. a page. privileged.

- soft physical address range. allocated by the module for further space. aligned on a page boundary.

- broadcast address range. Mirrors the system bus range. A write to a register in that range applies to all corresponding registers of a module on the same bus.

modules are processors, memory and I/O cards.

we could keep processor stats in the HPA space

memory module has entire memory as SPA space

I/O Address Space

As shown in the overall physical memory layout, the physical memory address range dedicated to I/O modules is located from `0x00_F000_0000` to `0x00_FFFF_FFFF`. The following figure depicts the I/O memory address space ranges in more detail.s

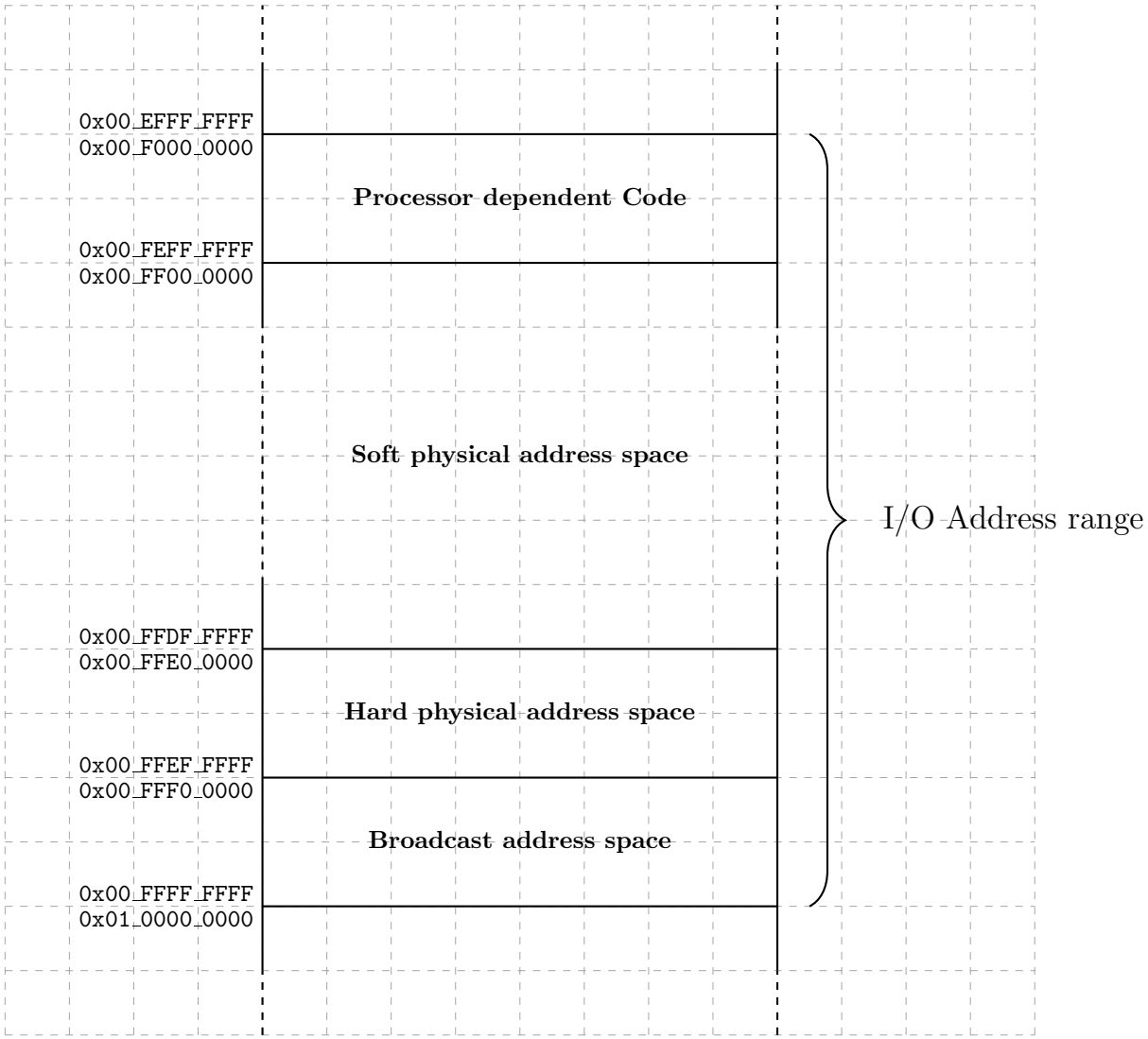


Figure 14: I/O Memory Address Range

The I/O address range is a 256Mbyte area in the first 4 Gbyte segment, i.e.segment 0. The area is divided into several ranges. The first range contains the processor dependent code. There is a set of library functions to manage the processor itself but also provide low-level primitives for the boot process and the operating system.

Hard physical address range

The **hard physical address** range contains a I/O page for each I/O modules installed. There is room for 64 I/O modules.

address is fixed by hardware, e.g a bus slot or so ...

Broadcast address range

The **broadcast address** range mirrors the hard physical address range. However, a write to these locations is a write operation to which all I/O modules will react.

Only write operations, read operations make no sense.

Soft physical address range

The **soft physical address**range is an area where space is allocated to an I/O module. The I/O module reacts to a memory access operation in its configured soft physical address range.

can be in physical memory and I/O address space

I/O Elements

Modules consist of I/O elements, smallest I/O unit of operaion

I/O elements are 128 bytes, i.e. 16 regs

I/O elements can also be two pages, one priv and one user level, mapped...

Part V

Runtime Environment

Runtime Overview

No CPU architecture with an idea of a runtime environment. Although the instruction set can be used to implement many models of runtime environments, there are common principles that exploit the CPU architecture. In fact, several CPU concepts were selected with a runtime already in mind. This chapter will present the Twin-64 runtime environment and describe the high level system model, the register usage convention, the execution model and calling convention.

System Environment

The following figure depicts a high-level overview of the software environment for the Twin-64 system. At the center of the execution model is the execution thread, referred to as a **task**. A task consists of the currently running execution object together with its private task-local data area, including its own stack and runtime context. All tasks of a job share a common job data area. At the highest level is the **system**, which contains the global data area for the system.

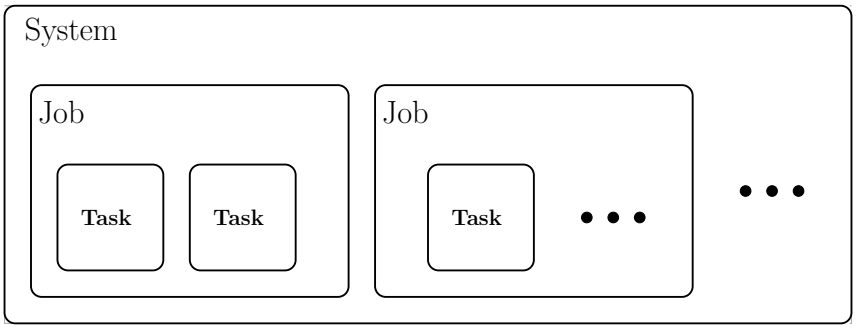


Figure 15: System Environment

All tasks belonging to the same job share a common job data area, which is used to store information and resources that must persist for the lifetime of the job and be visible across all of its tasks. A job may represent an interactive sessionsuch as a terminal sessionor it may be a non-interactive batch operation.

The system level contains the global data area, system libraries, and all services provided by what is traditionally called the operating system. The system is responsible for creating jobs, managing their lifetime, and scheduling the tasks within them.

The number of active tasks at any moment typically corresponds to the number of available processors in the system, allowing the system to take full advantage of available parallelism. While each task maintains its own private data and execution state, tasks within a single job share access to the job data area. All tasks, regardless of job, also have access to system-level services and global resources.

Runtime Environment

The processor is a general-purpose CPU. The runtime model presented here represents one possible implementation strategy, chosen to support the calling conventions and execution model that will be described in later sections. The following figure depicts a high-level overview of the runtime environment.

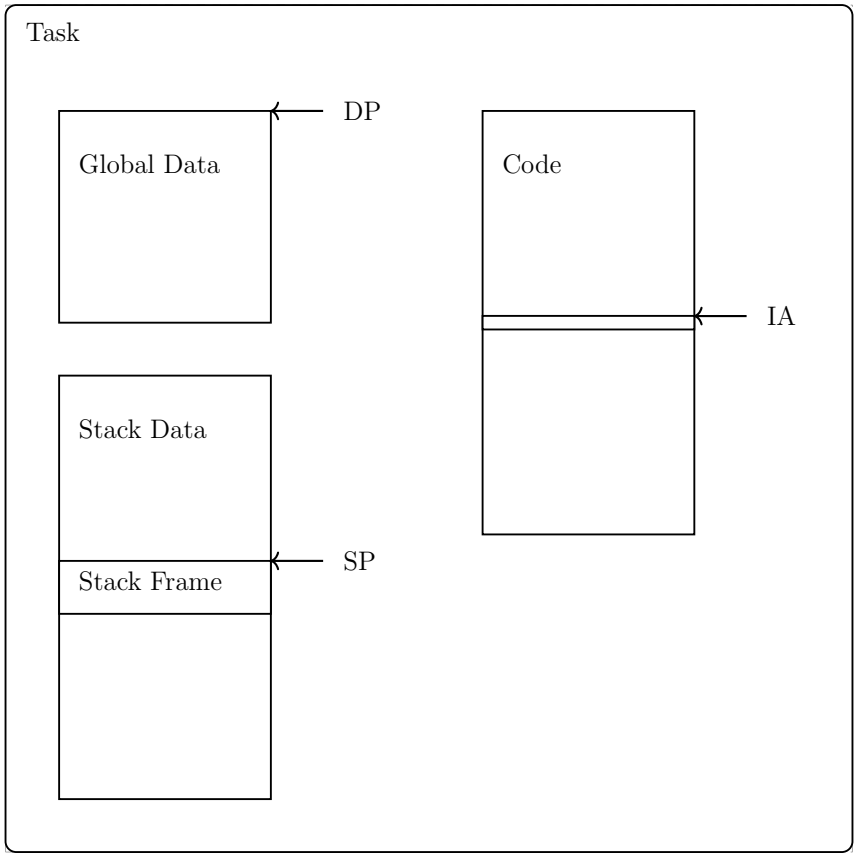


Figure 16: Runtime Environment

The code region contains program text that, once loaded, is immutable. Code is organized into modules, and a single task may reference several modules in its code region. The currently executing instruction is identified by the instruction address (IA).

Each code module provides a module-level global data section. The task-level global dataformed by aggregating the global sections of all modules referenced by the taskis addressed via the data pointer (DP). The data region also contains the stack area, which holds stack frames created by function calls. The active frame is located via the stack pointer (SP), and items within the stack are addressed relative to SP.

In this design the stack grows upward: new frames occupy increasing addresses, and older frame contents are typically referenced using expressions of the form `SP - <ofs>`. Each stack frame includes a frame marker that stores the previous SP and the return link (RL), enabling structured returns and proper unwinding of frames.

Note.

Register Usage

The CPU features general registers and control registers. All general registers can do computation as well as address computation. The runtime architecture builds on the general register file and assigns specific meaning to each register.

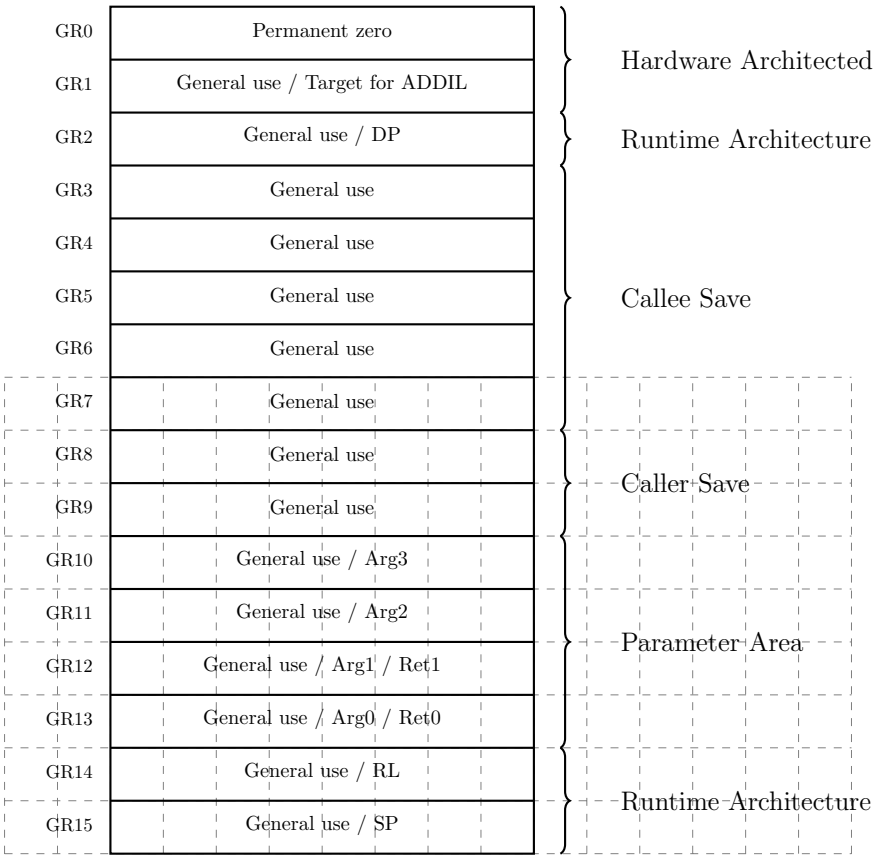


Figure 17: General Registers

Registers `R0` and `R1` are hardware-architected and have fixed, predefined roles within the processor. The remaining registers, `R2` through `R15`, form the general-purpose register file, whose usage is defined by the runtime environment. Within this convention, register `R2` is assigned as the data pointer (DP). It serves as the base reference for accessing the tasks global data region.

Registers `R3` through `R7` form the callee-save register set. A callee-save register is one whose value must be preserved by the called function: if the function uses such a register, it must save its original contents upon entry and restore them before returning. This ensures that long-lived values in the caller remain intact across function calls.

Registers `R8` and `R9` are designated as caller-save registers. A caller-save register may be overwritten by the called function; therefore, the caller is responsible for saving the registers value before making a call if it must be preserved.

Argument passing is handled through registers `R10` to `R13`, known as the parameter registers. When a function is invoked, its first four arguments are placed into these registers. Registers `R12` and `R13` also serve as the return-value registers, allowing a function to return up to two values without using memory.

Control flow is supported by register `R14`, the return-link register (RL). During a function call, the CALL instruction stores the return address in `R14`, allowing the function to return control to its caller.

Finally, register `R15` serves as the stack-frame pointer (SP). It marks the base of the current stack frame, which contains local variables, spilled registers, and temporaries, and supports predictable stack-based data management within the tasks runtime environment.

Stack Layout

Most modern programming languages rely on a **stack** to manage function calls, local variables, and return information. This stack is organized into **stack frames**, each representing the state of an active function invocation. When a function is called, a new frame is pushed onto the stack; when it returns, that frame is popped. This structured approach allows programs to keep track of nested calls efficiently and ensures that each function has its own isolated workspace during execution.

Stack Frame

A stack frame consists of four main areas, depicted in the following figure.

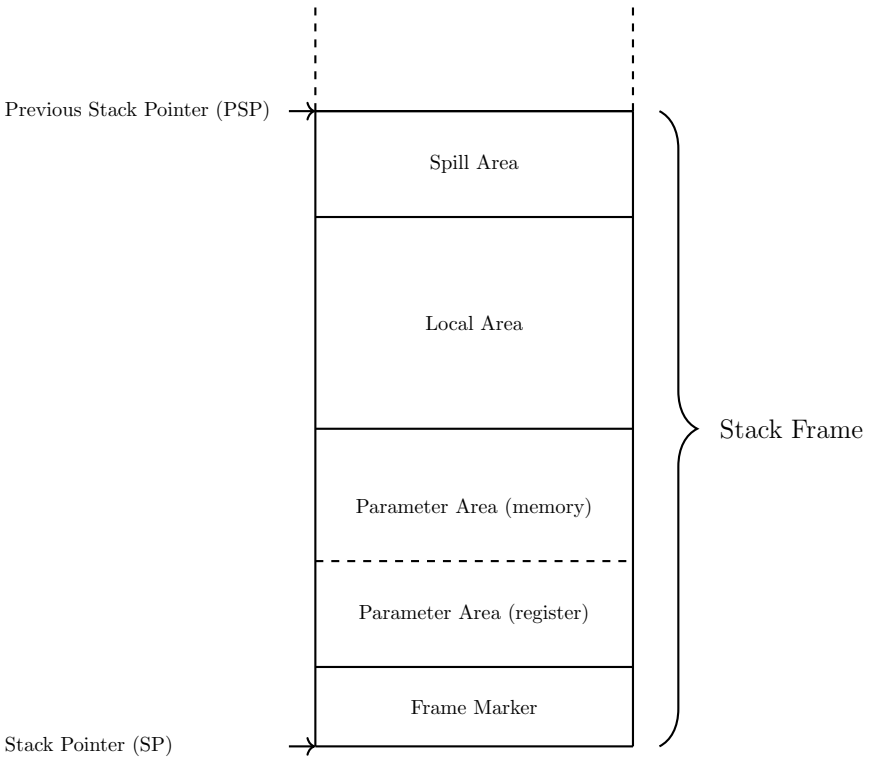


Figure 18: Stack Frame

The **stack frame marker** holds a pointer to the previous stack frame along with the return link address, allowing the call chain to be restored during function returns.

The **parameter area** is used for passing arguments to the called function and for returning data. Although the first four parameters are passed in registers, space for them must still be reserved in memory. Any parameters beyond the first four are placed directly in this area.

The **local area** stores the functions local variables. This is the region the function uses for data that must persist throughout its execution.

Finally, the **spill area** is used to save registers whose values must be preserved and restored before returning to the caller.

Stack Frame Addressing

All data items within a stack frame are accessed **relative to the current stack pointer (SP)**. The architecture uses an `SP - offset` addressing scheme, where each region of the stack frame—the frame marker, parameter area, local data area, and spill areais located at a fixed negative offset from the SP at function entry. Because the stack **grows upward in memory**, increasing addresses correspond to deeper regions of the current frame. This convention allows the compiler and debugger to compute the location of every stack-resident object from the SP alone, without requiring a dedicated frame pointer. As long as the SP remains stable during the execution of the function, all stack frame items can be reliably referenced through their predefined offsets.

Stack Frame Marker

A stack frame marker consists of two double words. The frame marker is pointed by the current stack pointer register (SP).

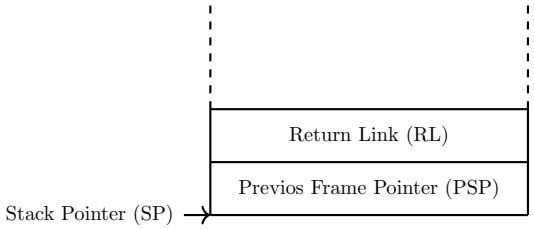


Figure 19: Stack Frame Marker

The first word holds the **address of the previous stack frame**. This field does not necessarily store the literal address, as the frame size can often be inferred from the instructions that created the current frame together with symbolic debugging information in the program image.

The second word contains the **return link**, which is the address immediately following the branch instruction in the calling function. If the called function is a leaf function, there is no need to save the return link in memory.

Parameter Area

A stack frame includes a **parameter area** large enough to hold the largest argument list passed from the calling function to the callee. Although the first four parameters are normally passed

in registers, corresponding space is still reserved in the frame to provide a uniform layout and to support cases where the callee needs to spill register arguments to memory. Any additional parameters beyond the first four are passed directly in the memory locations immediately below this register-parameter area. If a function neither receives parameters nor returns data, the parameter area is omitted from its stack frame, avoiding unnecessary stack usage.

Local Data Area

Immediately below the parameter area lies the **local data area**, which holds all automatic (local) variables of the function. These variables exist only for the duration of the function call and are allocated anew for each invocation. The compiler determines the size and placement of each local variable within this area, taking into account alignment requirements and optimization opportunities. If a function declares no local variables, this region has size zero and is omitted from the final stack frame layout.

Spill Area

The **spill area** contains any registers that the function must temporarily save in order to preserve the calling convention. Registers that must remain unchanged across function calls are saved ("spilled") into this area upon entry and restored before returning to the caller. The compiler allocates spill slots only when necessary; if a function does not need to save any registers, the spill area is not present in its stack frame. This selective allocation helps keep stack usage minimal while maintaining correct program behavior.

Calling Convention

The previous chapters introduced the register usage conventions and the overall stack layout defined by the runtime architecture. Building on that foundation, this chapter describes the mechanisms used to invoke routinesboth simple procedures and more substantial functionsand the conventions that govern control transfer, parameter passing, register preservation, and stack-frame management.

The discussion begins with the simplest form of call, the local call, which transfers control between routines within the same module. Local calls illustrate the fundamental steps of the calling convention without the additional considerations required for inter-module calls.

A routine call proceeds through several well-defined stages. First, the caller evaluates all argument expressions and places the resulting values into the parameter area, either in registers or on the stack as required. After preparing the parameters, the caller saves any registers whose values must remain intact across the call. These caller-save registers must be preserved by the caller before branching to another routine.

The called routinethe calleeaccesses the incoming parameters in the established layout of the parameter area and begins execution at its entry point. Upon entry, the callee allocates a stack frame, records the return link, and saves any callee-save registers it intends to overwrite. This setup ensures that both caller and callee operate independently while maintaining a consistent runtime state.

Once the callee completes its computation, it prepares to return. All callee-save registers are restored, the return address is retrieved from the frame marker, and control is transferred back to the caller. This division of responsibility allows efficient cooperation between routines and avoids unnecessary state preservation.

As described earlier, the general-purpose register set is partitioned into caller-save and callee-save regions. A compiler selects registers from these regions as needed, balancing the work of saving and restoring state between the caller and the callee. When possible, redundant saves and restores are eliminated. The runtime architecture distinguishes between two basic categories of calls: local calls, which remain within a single module, and external calls, which cross module boundaries and require additional handling. The following sections describe the two types in detail.

Local Call

A local call is by far the most common type of procedure call. The following code snippets illustrate such a call. The calling procedure prepares the call by loading the arguments, saving any registers that the callee might modify, and then branching to the callee. After the callee returns, the caller restores its saved registers and continues execution.

```
1
2 ; calling procedure
3             ...                 ; load arguments
4             ...                 ; save caller saves
5             BL target, RL       ; branch to "target"
6             ...                 ; restore caller saves
```

The called procedure begins by saving the return link, allocating its stack frame, and saving any registers that the caller expects to remain unchanged across the call. At the end of the routine, it restores the return address, restores any callee-saved registers, deallocates the stack frame, and finally branches back to the caller using the restored return address.

```
1
2 ; called procedure
3
4 target:     ...                 ; save RP
5             ...                 ; allocate stack frame
6             ...                 ; save callee save
7
8             ...                 ; do the work
9
10            ...                 ; restore RP
11            ...                 ; restore callee saves
12            ...                 : deallocate stack frame
13            BV (RL)             ; return to caller
```

The code above shows the full sequence of steps that may be required for a local procedure call. Depending on the specific call and the nature of the callee, some steps may be omitted. For example, if the caller does not use any caller-saved registers, there is no need to save them before the call. Likewise, a procedure with no arguments does not need to evaluate arguments.

The callee can also omit unnecessary steps. A leaf procedurethat is, a procedure that does not call any other proceduredoes not need to save the return address. If it does not use any callee-saved registers, those need not be saved or restored. Finally, if a leaf procedure uses no caller-saved registers, has no local variables, and does not need to save any temporary registers, it may not require a stack frame at all.

The instructions `B`, `BR` and `BV` are the instructions used for a local. The address computation, whether IA-relative or region relative, will not impact the region portion of the instruction address.

Parameter passing

Procedures optionally pass arguments to a called procedure and obtain return values. The parameter area is located in the calling procedure's stack frame below the stack frame marker.

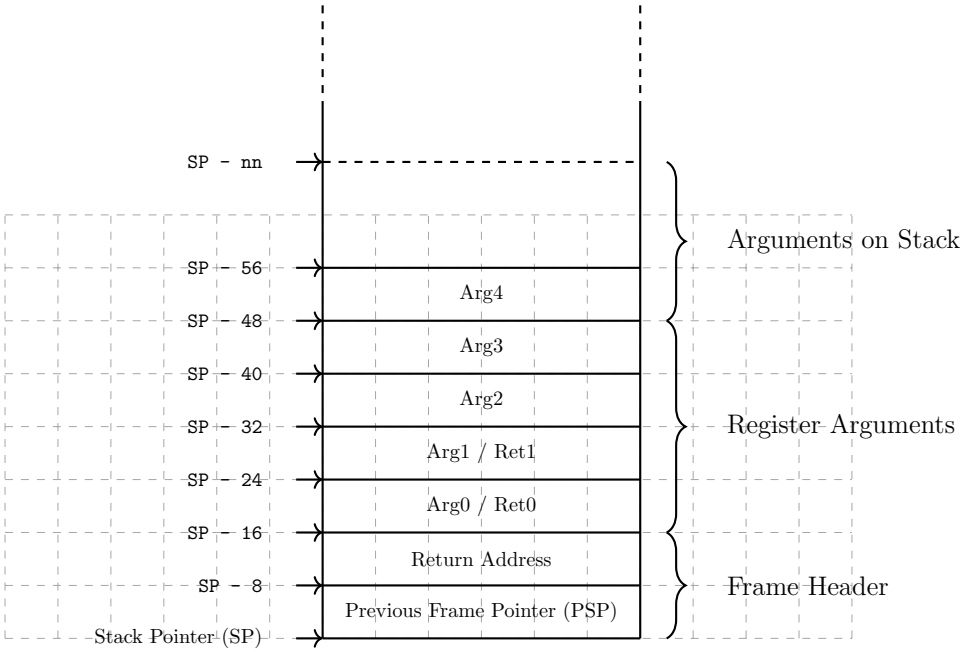


Figure 20: Argument Passing

The first four arguments are passed via the registers ARG0 to ARG3. Nevertheless, the stack frame parameter area allocates stack space for them. Any further parameter is stored in the stack frame parameter area. Upon return, the called procedure may pass up to two return values in registers, `Ret0` and `Ret1`. Arguments and return values are addressed relative to the stack frame pointer of the calling procedure.

External Calls

An external call is defined as potentially crossing the region boundary. The `BE` instruction computes the effective address from a general register and an immediate offset encoded in the instruction. In contrast to the local branch type instruction, the general register used contains the virtual address and not only the offset portion.

??? ..... ???

Linkage Table

each module as a set of entries in the linkage table for the external references of this module.

The assembler / compiler generates the linkage table skeleton.

an external procedure has an entry, also a procedure label

??? replace by instruction64 style...

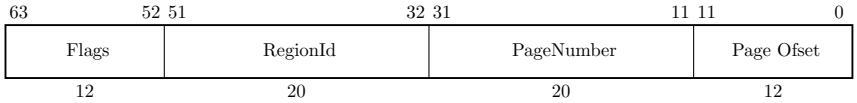


Figure 21: Processor Configuration Register

flags need to be defined, important that the `BE` instruction always ignores the upper 12 bits.

table is at an architected location on the current region, offset zero.

the offset in the `BE` instruction refers to the entry. This allows for 16384 entries considering a one double word entry.

`offset` is the index into the linkage table

External Call

The external address is taken from the linkage table entry. The offset was computed at compile time.

```
1            .
2            .
3            .
4            .
```

An exported procedure will have a different entry and ext sequence compared to a local procedure. The .ENTER and .LEAVE pseudo instructions will generate the respective code sequences.

The .CALLINFO pseudo instruction will describe the procedure.

The code snippet below is the maximum code to be generated.

```
1            .CALLINFO ...       ; procedure description
2
3            ;ENTER sequence
4            ST SP,(SP)          ; save PSP
5            ST RL, -8(SP)       ; save RL
6            ST DP, -16(SP)      ; save DP
7            MFIA.S R1           ; get module LTAB address
8            LD DP, <ofs>(R1)    ; load module DP
9            LDO SP,<size>(SP)   ; allocate stack frame
10           .
11           .
12           .
13           .
14           ;LEAVE sequence
15           LDO SP, -<size>(SP) ; deallocate stack frame
16           LD  DP, -16(SP)     ; load previous DP
17           LD  RL, -8(SP)      ; load return link
18           BE (RL)             ; external branch return
```

the DP value is taken from the first entry in the LTAB section for the module.

for privilege level changes, there needs to be a stub on a gateway page.

```
1               ; on a gateway page
2               .
3               .
4               B local target ;jump to the target procedure.
5               .
6               .
```

to investigate: is a simple branch enough ? should it rather be a jump based on an entry in an export table ? LTAB alike ?

Part VI

Processor Design

Processor Design

xxx

Part VII

Simulator Environment

Simulator

xxx

Part VIII

Appendix

Appendix - Instruction Commentary

This appendix presents extended commentary on selected instruction groups. It provides further explanation, design rationale, and hardware-level insights that complement the main instruction descriptions. Each subsection discusses the motivation, encoding, and implementation strategy for one or more related instructions, offering a deeper view into the processors architectural design.

list what is ahead...

Arithmetic Operation Instructions

ADD, SUB, SHLxA, SHRxA

Bit Operation Instructions

AND, OR, XOR, EXTR, DEP, DSR

Comparison and Branch Condition Encoding

This appendix explains the condition encoding and implementation philosophy used by the `CMP`, `CBR`, `ABR`, and `MBR` instructions. The design goal is to minimize hardware complexity by reusing a single subtraction/comparator path. Comparison operators are encoded in a compact and symmetric way that makes condition inversion trivial.

The following instructions use the same 3-bit comparison field (`cond`):

- **CMP** Compares two operands using the 3-bit `cond` field and produces a boolean result (0/1) in a register.
- **CBR** Compares two registers and branches based on the same 3-bit `cond` field. This performs a direct comparison without requiring a preceding `CMP`.
- **ABR** Performs an add operation and branches using a 3-bit `cond` field that tests the *add result* against zero or other simple predicates. Intended for loop counters or pointer walking.
- **MBR** Performs a move operation and branches using the same 3-bit `cond` field. The comparison is applied to the source operand, typically against zero.

All four instructions share a uniform 3-bit condition encoding, allowing common decoder and comparator logic. Branch target computation uses a sign-extended 15-bit offset shifted left by two, allowing word-aligned branch destinations within a `64 KB` range relative to the current instruction address.

Design Rationale The condition encoding is designed for symmetry and minimal logic cost. Conditions are arranged in complementary pairs so that flipping a single bit yields the inverse relation. This symmetry makes branch inversion trivial in hardware and simplifies assembler implementation.

Value	Binary	Mnemonic	Meaning / Branch if
0	000	`EQ`	Equal (`==`)
1	001	`LT`	Less than (`<`)
2	010	`GT`	Greater than (`>`)
3	011	`EV`	Even (bit 0 = 0)
4	100	`NE`	Not equal (`!=`)
5	101	`GE`	Greater than or equal (`>=`)
6	110	`LE`	Less than or equal (`<=`)
7	111	`OD`	Odd (bit 0 = 1)

The most significant bit (`cond[2]`) acts as an *invert bit*: flipping it inverts the sense of the comparison. The relationships between base and inverted conditions are as follows:

Base Condition	Inverted Condition	Encoding Pair
`EQ`	`NE`	000 / 100
`LT`	`GE`	001 / 101
`GT`	`LE`	010 / 110
`EV`	`OD`	011 / 111

This arrangement keeps decoding logic compact and uniform across all instruction forms.

Logical Decoding and Hardware Signals All conditions used by the comparator and branch unit are derived from simple signals computed by the ALU after a subtraction, or directly from the result when testing against zero. The example below illustrates a possible Verilog implementation:

```
wire Z  = (result == 64'd0);  // Zero flag
wire S  = result[63];         // Sign flag (for signed comparisons)
wire B0 = result[0];          // Least significant bit (for EV/OD)

wire cond_EQ =  Z;
wire cond_LT =  S;
wire cond_GT = (~S) & (~Z);
wire cond_EV = ~B0;

wire cond_NE = ~Z;
wire cond_GE = ~S;
wire cond_LE =  S | Z;
wire cond_OD =  B0;

assign branch_taken =
    (cond == 3'b000) ? cond_EQ :
    (cond == 3'b001) ? cond_LT :
    (cond == 3'b010) ? cond_GT :
    (cond == 3'b011) ? cond_EV :
    (cond == 3'b100) ? cond_NE :
    (cond == 3'b101) ? cond_GE :
    (cond == 3'b110) ? cond_LE :
                       cond_OD;
```

All comparisons are signed by default; unsigned comparisons can be implemented by treating operands as unsigned integers or through a future `CMPU` variant if required. The `EV`/`OD` conditions allow efficient branching on bit 0, which is useful for address alignment tests, distinguishing even/odd indices, and loop unrolling.

Because all comparison and branch instructions share this common encoding and logic, the branch unit requires only a single multiplexer and no dedicated condition register, simplifying both the datapath and verification.

Control Flow Instructions

BB, B, BR, BV, BE

Immediate Instructions

ADDIL, LDI, LDO

Memory Reference Instructions

LD, ST

but also: ADD, SUB, AND, OR, XOR, CMP

post increment strategy

Load-Reserved and Store-Conditional Instructions

The `LDR` (Load Reserved) and `STC` (Store Conditional) instructions provide the architectural mechanism for implementing atomic read-modify-write sequences without requiring explicit locks. Together, they form the foundation for synchronization primitives such as semaphores, spinlocks, and atomic counters.

- **LDR** loads a 64-bit value from memory and establishes a *reservation* on the cache line containing that address. The effective address is computed as the sum of the base register and the signed offset and must be 8-byte aligned.
- **STC** attempts to store the value in `Rn` to the same memory location, succeeding only if the reservation remains valid. If the reservation is valid, `STC` completes normally and returns a success indication (typically a value of 1). If the reservation has been lost, the store is suppressed and a failure value (0) is returned to the destination register.

Design Rationale The `LDR`/`STC` pair allows atomic sequences to be expressed entirely in software, avoiding pipeline stalls and bus locks. The mechanism relies on a single internal reservation register that tracks the address (or cache line) of the most recent `LDR`. Only one reservation may be active per processor at any time. Using the cache system as the underlying reservation tracker reduces hardware cost and ensures natural invalidation under normal cache-coherency events. A reservation is cleared automatically under any of the following conditions:

- The cache line containing the reserved address is invalidated or evicted through the local or other processors.
- The local processor performs a write to the same cache line through any path other than `STC`.
- An interrupt, trap, or context switch occurs.
- A subsequent `LDR` instruction establishes a new reservation.

This model guarantees that at most one `STC` can succeed per `LDR`, ensuring forward progress while maintaining strong atomicity at the cache-line granularity.

Implementation Notes The reservation register may store the cache line tag rather than the full address, allowing fast comparison against cache operations. Integration with the cache controller ensures that any coherence or invalidation event automatically clears the reservation flag. Because interrupts also clear reservations, interrupt latency and atomicity properties remain predictable.

This design matches the behavior of similar primitives in other architectures (e.g., RISC-V `LR/SC` or PowerPC `lwarx/stwcx.`) but uses a cache-linebased validity rule rather than a fixed address match, simplifying coherence handling in multiprocessor systems.

Atomic Increment Example The following example implements an atomic increment function. The memory location is specified by base register GR5 and an offset.

```
atomic_inc:
    LDR      R1, ofs( R5 )     ; Load current value and set reservation
    ADDI     R1, R1, 1         ; Increment locally
    STC      R1, ofs( R5 )     ; Attempt conditional store
    CBR.EQ   R1, R0, atomic_inc    ; Retry if store failed (R1 = 0)
    RET
```

Each iteration performs a reserved load, modifies the value, and attempts a conditional store. If another processor writes to the same cache line between the `LDR` and `STC`, the reservation is cleared, causing `STC` to fail and the loop to retry. This ensures that the final store reflects an atomic update to memory.

Compare-and-Swap (CAS) Example The following example implements the compare and swap function.

```
;   Arguments:
;      Arg0   : base address of the value location
;      Arg1   : expected old value
;      Arg2   : desired new value
;
;   Returns:
;      Arg0  : 1 on success, 0 on failure

cas:
    LDR    R1, 0(Arg0)           ; load reserved value
    CBR.EQ R1, Arg1, cas_fail    ; compare with expected value
                                 ; If not equal, fail
    STC    Arg2, 0(Arg0)         ; Attempt conditional store
    MBR    Arg2, Arg2, cas       ; Retry if store failed
    OR     Arg0, R0, 1           ; Indicate success
    BV ( R2 )                    ; return

cas_fail:
    OR     Arg0, R0, 0           ; Indicate failure
    BV ( R2 )                    ; return
```

This CAS sequence allows software-based atomic updates: the memory location is only updated if its current value matches the expected value. Failed stores due to lost reservations cause the loop to retry, guaranteeing atomicity even in multiprocessor environments.

Summary Together, these examples demonstrate how the `LDR`/`STC` mechanism enables fully software-based atomic readmodifywrite operations. By combining reserved loads with conditional stores, the processor can implement synchronization primitives, spinlocks, and lock-free data structures without additional hardware beyond the reservation register and cache-line tracking.

Appendix - Instruction Notation Routines

describe the routines used in the instruction set operation part...

Table 11: Instruction Notation Routines

Function	Description
addOfs32(base, ofs)	...
...	...

Appendix - Programming Notes

xxx

Appendix -TLB and Caches

??? perhaps add to processor appendix...

Appendix -Diagnostic Functions

xxx

Table 12: Diagnostic Functions

Id	Function	Arg0	Arg1
0	No operation	0	0
1	Write status display	value	-

Appendix - Glossary

Address Space A logical domain of addresses. The CPU may define multiple address spaces, such as *main memory space* and *I/O space*. Each space defines an independent address range and access semantics.

Region A contiguous range of addresses within an address space, reserved for a particular purpose. Examples: *code region*, *data region*, *stack region*.

Object A unit of storage within a region. Objects correspond to program-visible entities or runtime allocations, such as global variables, heap blocks, or stack frames. Examples: *global object*, *heap object*, *stack object*.

ELF Section A named subdivision in an ELF file used by the linker and compiler (e.g., `.text`, `.data`, `.bss`). Sections are grouped by the linker into ELF segments.

ELF Segment A loadable portion of an ELF file (defined by a program header). Each ELF segment is mapped into one or more regions of an address space during program loading.

Mapped Region A runtime mapping that associates an ELF segment or memory source with a region in an address space. The emulator or loader tracks mapped regions to manage memory access and protection.